BerlinMOD Benchmark on MobilityDB

Esteban Zimányi

COLLABORATORS

	TITLE : BerlinMOD Benchmark on MobilityDB	

ACTION	*NAME*	*DATE*	*SIGNATURE*
WRITTEN BY	Esteban Zimányi	November 7, 2026	

REVISION HISTORY

NUMBER	DATE	DESCRIPTION	NAME

Contents

1 MobilityDB Tutorial **1**

1.1 Installation . . . 1
1.2 Loading the Data . . . 4
1.3 Loading the Data in Partitioned Tables . . . 6
1.4 Exploring the Data . . . 9
1.5 Querying the Data . . . 11
1.5.1 Range Queries . . . 11
1.5.2 Temporal Aggregate Queries . . . 12
1.5.3 Distance Queries . . . 13
1.5.4 Nearest-Neighbor Queries . . . 14

2 Generating Realistic Trajectory Datasets **17**

2.1 Introduction . . . 17
2.2 Contents . . . 17
2.3 Tools and Data . . . 18
2.4 Quick Start . . . 18
2.5 Exporting the Generated Data . . . 19
2.6 Exploring the Generated Data . . . 20
2.7 Understanding the Generation Process . . . 23
2.8 Customizing the Generator to Your City . . . 32
2.9 Tuning the Generator Parameters . . . 33
2.10 Changing the Simulation Scenario . . . 35
2.11 Creating a Graph from Input Data . . . 39
2.11.1 Creating the Graph . . . 40
2.11.2 Linear Contraction of the Graph . . . 44

3 BerlinMOD Benchmark on MobilityDB **48**

3.1 Loading the Data . . . 48
3.2 Loading the Data in Partitioned Tables . . . 52
3.3 BerlinMOD/r Queries . . . 53

List of Figures

1.1 Configuration of a connection to the docker image in pgAdmin. 3

1.2 Visualization of the trips in QGIS. The streets are shown in blue, the trips are shown in black, the home nodes in black and the work nodes in red. 10

2.1 Visualization of a long trip. 21

2.2 Assigning in QGIS a gradient color from blue to red according to the value of the attribute `count`. 22

2.3 Visualization of the edges of the graph according to the number of trips that traversed the edges. 23

2.4 Visualization of the edges of the graph according to the speed of trips that traversed the edges. 24

2.5 Defining the bounding box for obtaining OSM data from Barcelona. 33

2.6 Visualization of the data generated for the deliveries scenario. The road network is shown with blue lines, the warehouses are shown with a red star, the routes taken by the deliveries are shown with black lines, and the location of the customers with black points. 39

2.7 Visualization of the deliveries of one vehicle during one day. A delivery trip starts and ends at a warehouse and make the deliveries to several customers, four in this case. 40

2.8 Comparison of the nodes obtained (in blue) with those obtained by osm2pgrouting (in red). 44

2.9 Comparison of the nodes obtained by contracting the graph (in black), before contraction (in blue), and those obtained by osm2pgrouting (in red). 47

Abstract

MobilityDB is an extension to the PostgreSQL object-relational database system and its spatial extension PostGIS. It allows temporal and spatio-temporal objects to be stored in the database, that is, objects whose attribute values and/or location evolves in time. This document shows an implementation of the BerlinMOD benchmark that is described in:

> Düntgen, C., Behr, T. and Güting, R.H. BerlinMOD: a benchmark for moving object databases. The VLDB Journal 18, 1335 (2009). https://doi.org/10.1007/s00778-009-0142-5

It starts with a tutorial introducing MobilityDB based on BerlinMOD data, continues by explaining how to generate realistic trajectory datasets of arbitrary size, and concludes by explaining how to run the BerlinMOD benchmark on MobilityDB.

MobilityDB is open source and its code is available on Github. MobilityDB is developed by the Computer & Decision Engineering Department of the Université libre de Bruxelles (ULB) under the direction of Prof. Esteban Zimányi. ULB is an OGC Associate Member.

Chapter 1

MobilityDB Tutorial

To illustrate the capabilities of MobilityDB, we give an example use case that loads, explores, and query mobility data. The data used is based on the MobilityDB implementation of the BerlinMOD benchmark for moving object databases. The data is available as a ZIP file.

1.1 Installation

For this tutorial we can use two alternative installations:

- Install from sources
- Use a Docker image containing MobilityDB and all its dependencies (including PostgreSQL and PostGIS)

We explain first the installation from sources and later in this section we explain the installation using a Docker image.

In order to use this tutorial you must first have MobilityDB installed in your system. We refer to the MobilityDB documentation for doing this task. Then we can install the MobilityDB-BerlinMOD tutorial as follows.

```
git clone https://github.com/MobilityDB/MobilityDB-BerlinMOD.git
cd MobilityDB-BerlinMOD/BerlinMOD
```

We explain now how to explore this tutorial using a Docker image that contains MobilityDB and all its dependencies (including PostgreSQL and PostGIS). The container has a default database called `mobilitydb` with the MobilityDB extension installed where `user = pw = docker`. This presupposes that you have installed Docker into your computer. In that case, you can run the following command.

```
docker pull mobilitydb/mobilitydb
docker volume create mobilitydb_data
docker run --name "mobilitydb" -d -p 25432:5432 -v mobilitydb_data:/var/lib/postgresql
mobilitydb/mobilitydb
```

In the above commands

- `docker pull` downloads the Docker image of `mobilitydb`. If the image has been downloaded before, this checks whether a more recent image has been published in the docker repository, and downloads it. It is better to call this command every time, to ensure that you have the most up-to-date version of this image.
- `docker volume create mobilitydb_data` creates a volume container on the host, that we will use to persist the PostgreSQL database files outside of the MobilityDB container. You need to run this command only once, during the first use of the image
- `docker run --name=mobilitydb` tells Docker our new container will be named `mobilitydb`.

- `-d` runs the container in the background (detached mode).
- `-p 25432:5432` maps TCP port 5432 in the container to port 25432 on the Docker host (to prevent potential conflicts with any local database instance you may have). This is required because the PostgreSQL database server in the container listens for connections on port 5432 by default.
- `-v mobilitydb_data:/var/lib/postgresql` tells the container filesystem to mount the `mobilitydb_data` volume that we have just created to the path `/var/lib/postgresql`. This means that any database objects that the container saves or creates (by default in `/var/lib/postgresql`) will instead be persisted in the `mobilitydb_data` directory, which is stored in the host. This options ensures that your data will not be lost when the container is removed.
- `mobilitydb/mobilitydb` tells Docker to pull the docker image with that name from Docker Hub.

Now we can launch any PostgreSQL administrative front-end to start using MobilityDB. Two traditional ones are the command-line tool `psql` and the graphical tool `pgAdmin`. We can launch `psql` as follows.

```
docker exec -t -i mobilitydb psql -h localhost -p 5432 -d mobilitydb -U docker
```

In the above command

- `docker exec -t -i mobilitydb psql` tells Docker to allocate a pseudo-TTY, to keep STDIN open, and to execute in the container `mobilitydb` the command `psql`.
- `-h localhost -p 5432 -d mobilitydb -U docker` tells `psql`, respectively, the database server host, the server port, the database name, and the user name.

Note that you will be prompted to provide the password, which is also `docker`.

In order to launch `pgAdmin`, there are two options to create a connection. The first option is to set the host to the localhost (127.0.0.1), and the port to the mapped one on the host, as per the `docker run` command. In this example the port is 25432. Now we can launch `pgAdmin` and establish a new connection to the docker container. This is done as shown in Figure 1.1.

The second option is to know the IP address used by docker container with the following command.

```
docker-machine ip
-- 192.168.99.101
```

Notice that the address obtained in your computer may be different from the one above. Now we can launch `pgAdmin` and establish a new connection to the docker container. This is done as shown in Figure 1.1. The second option is to set the host to the localhost (127.0.0.1), and the port to the mapped one on the host, as per the `docker run` command. In this example the port would be 25432.

Now you can use pgAdmin to query the mobilitydb database, as will be further explained in the following sections. Here are few more docker commands that you will eventually need:

```
docker stop "mobilitydb"
docker start "mobilitydb"
docker rm "mobilitydb"
```

The above commands

- `docker stop` shuts down the docker container. You need to issue this command, for example, if you need to re-start the host.
- `docker start` launches back the docker container. You need to issue this command, for example, after re-starting the host.
- `docker rm` removes/deletes docker container. You need to issue this command, for example, if you need to `docker pull` a more recent MobilityDB image. If the databases are stored in a `docker volume` as explained above, it will still be available after downloading and running the new image.

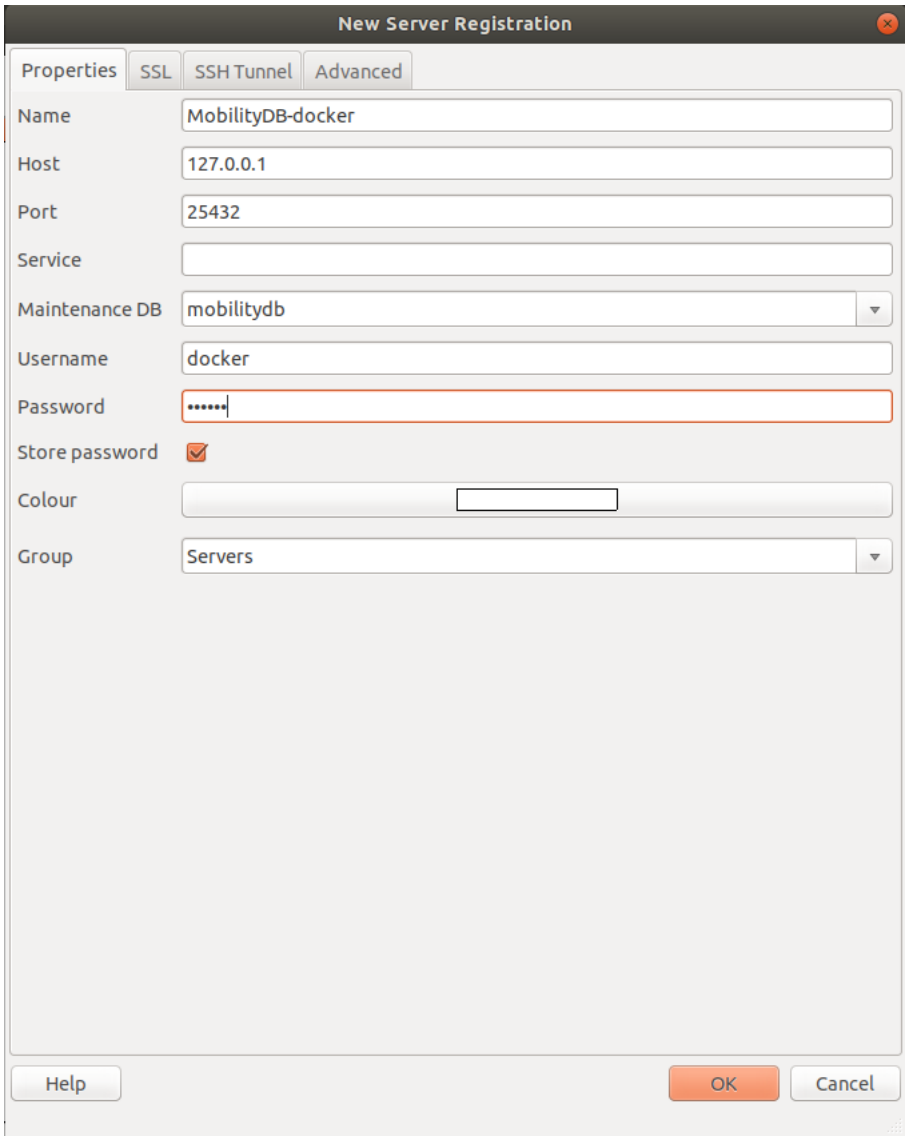

Figure 1.1: Configuration of a connection to the docker image in pgAdmin.

1.2 Loading the Data

The ZIP file with the data for this tutorial contains a set of CSV files as follows:

- `instants.csv` with fields `InstantId` and `Instant` contains timestamps used for queries.
- `licences.csv` with fields `LicenceId`, `Licence` and `VehicleId` contains vehicle licences used for queries.
- `periods.csv` with fields `PeriodId`, `Begin`, and `End` contains periods used for the queries.
- `points.csv` with fields `PointId`, `PosX`, and `PosY` contains points used for queries.
- `regions.csv` with fields `RegionId`, `PointNo`, `PosX`, and `PosY` and `Yend` contains the polygons used for queries.
- `trips.csv` with fields `TripId`, `VehicleId`, `PosX`, `PosY`, and `Instant` contains vehicles movements and pauses.
- `vehicles.csv` with fields `VehicleId`, `Licence`, `VehicleType`, and `Model` contains the vehicle descriptions.

We decompress the file with the data into a directory. This can be done using the command.

```
unzip berlinmod_data.zip
```

We suppose in the following that the directory used is as follows `/home/mobilitydb/data/`.

In the following, we can use the `mobilitydb` database provided in the container. This database has already installed the MobilityDB extension. Alternatively, you may use another database. In that case, you can install the MobilityDB extension in your database by using the following command.

```
CREATE EXTENSION MobilityDB CASCADE;
```

By using `CASCADE` we load the required PostGIS extension prior to loading MobilityDB.

We create the tables to be loaded with the data in the CSV files as follows.

```
CREATE TABLE Instants (
  InstantId integer PRIMARY KEY,
  Instant timestamptz NOT NULL );
CREATE TABLE Periods (
  PeriodId integer PRIMARY KEY,
  Tstart TimestampTz NOT NULL,
  Tend TimestampTz NOT NULL,
  Period tstzspan );
CREATE TABLE Points (
  PointId integer PRIMARY KEY,
  PosX double precision NOT NULL,
  PosY double precision NOT NULL,
  Geom Geometry(Point) );
CREATE TABLE RegionsInput (
  RegionId integer,
  PointNo integer,
  XPos double precision NOT NULL,
  YPos double precision NOT NULL,
  PRIMARY KEY (RegionId, PointNo) );
CREATE TABLE Regions (
  RegionId integer PRIMARY KEY,
  Geom Geometry(Polygon) NOT NULL );
CREATE TABLE Vehicles (
  VehicleId integer PRIMARY KEY,
  Licence text NOT NULL,
  VehicleType text NOT NULL,
  Model text NOT NULL );
CREATE TABLE Licences (
  LicenceId integer PRIMARY KEY,
```

```
  Licence text NOT NULL,
  VehicleId integer NOT NULL REFERENCES Vehicles(VehicleId) );
CREATE TABLE TripsInput (
  TripId integer NOT NULL,
  VehicleId integer NOT NULL REFERENCES Vehicles(VehicleId),
  PosX float NOT NULL,
  PosY float NOT NULL,
  t timestamptz NOT NULL,
  PRIMARY KEY (TripId, t) );
CREATE TABLE Trips (
  TripId integer PRIMARY KEY,
  VehicleId integer NOT NULL REFERENCES Vehicles(VehicleId),
  Trip tgeompoint NOT NULL );
```

We created one table for each CSV file. In addition, we created a table `Regions` in order to assemble all points composing the polygon of a region into a single geometry and a table `Trips` in order to assemble all instants composing a trip into a single temporal point.

We can load the CSV files into the corresponding tables as follows.

```
COPY Instants(InstantId, Instant) FROM '/home/mobilitydb/data/instants.csv'
  DELIMITER ',' CSV HEADER;
COPY Periods(PeriodId, Tstart, Tend) FROM '/home/mobilitydb/data/periods.csv'
  DELIMITER ',' CSV HEADER;
UPDATE Periods
SET Period = span(Tstart, Tend);
COPY Points(PointId, PosX, PosY) FROM '/home/mobilitydb/data/points.csv'
  DELIMITER ',' CSV HEADER;
UPDATE Points
SET Geom = ST_Transform(ST_SetSRID(ST_MakePoint(PosX, PosY), 4326), 5676);
COPY RegionsInput(RegionId, PointId, XPos, YPos) FROM
  '/home/mobilitydb/data/regions.csv' DELIMITER  ',' CSV HEADER;
COPY Vehicles(VehicleId, Licence, VehicleType, Model)
  FROM '/home/mobilitydb/data/vehicles.csv' DELIMITER ',' CSV HEADER;
COPY Licences(LicenceId, Licence, VehicleId) FROM '/home/mobilitydb/data/licences.csv'
  DELIMITER ',' CSV HEADER;
COPY TripsInput(TripId, VehicleId, PosX, PosY, t) FROM '/home/mobilitydb/data/trips.csv'
  DELIMITER ',' CSV HEADER;
```

The following query is used to load table `Regions` from the data in table `RegionsInput`.

```
INSERT INTO Regions(RegionId, Geom)
SELECT RegionId, ST_MakePolygon(ST_MakeLine(array_agg(
  ST_Transform(ST_SetSRID(ST_MakePoint(PosX, PosY), 4326), 5676) ORDER BY PointNo)))
FROM RegionsInput
GROUP BY RegionId;
```

There are many nested functions, so reading from the innermost:

- Function `ST_MakePoint` construct a point from the `PosX` and `PosY` values.
- Function `ST_SetSRID` sets the SRID of the point to 4326, that is, to the standard WGS84 GPS coordinates.
- Function `ST_Transform` transforms the spherical GPS coordinates to plannar coordinates fitted for Belgium.
- Function `array_agg` collects in an array all points of a region (as specified by the `GROUP BY` clause) and sort them by `PointNo` (as specified by the `ORDER BY` clause).
- Function `ST_MakeLine` make a linestring from the array of all points in a region.
- Function `ST_MakePolygon` make a polygon for the region from a linestring.

The following query is used to load table `Trips` from the data in table `TripsInput`.

```
INSERT INTO Trips(TripId, VehicleId, Trip)
SELECT TripId, VehicleId, tgeompoint_seq(array_agg(tgeompoint_inst(
  ST_Transform(ST_SetSRID(ST_MakePoint(PosX, PosY), 4326), 5676), t) ORDER BY t))
FROM TripsInput
GROUP BY VehicleId, TripId;
```

There are many nested functions, so reading from the innermost:

- Function `ST_MakePoint` construct a point from the `PosX` and `PosY` values.
- Function `ST_SetSRID` sets the SRID of the point to 4326.
- Function `ST_Transform` transforms the spherical coordinates to plannar coordinates with SRID 5676.
- Function `tgeompoint_inst` gets the point and the time values to create a temporal point of instant duration.
- Function `array_agg` collects in an array all temporal instant points of a given vehicle and a given trip (as specified by the `GROUP BY` clause) and sort them by time (as specified by the `ORDER BY` clause).
- Function `tgeompoint_seq` gets the array of temporal points and construct a temporal sequence point.

Finally, we create indexes on traditional, spatial, temporal or spatiotemporal attributes as well as views to select a subset of the rows from the corresponding tables. This can be done as follows.

```
CREATE INDEX Instants_Instant_Idx ON Instants USING btree(Instant);
CREATE INDEX Periods_Period_Idx ON Periods USING gist(Period);
CREATE INDEX Points_Geom_Idx ON Points USING gist(Geom);
CREATE INDEX Regions_Geom_Idx ON Regions USING gist(Geom);
CREATE INDEX Trips_VehId_Idx ON Trips USING btree(VehicleId);
CREATE INDEX Trips_Trip_gist_Idx ON Trips USING gist(trip);

CREATE VIEW Instants1 AS SELECT * FROM Instants LIMIT 10;
CREATE VIEW Periods1 AS SELECT * FROM Periods LIMIT 10;
CREATE VIEW Points1 AS SELECT * FROM Points LIMIT 10;
CREATE VIEW Regions1 AS SELECT * FROM Regions LIMIT 10;
CREATE VIEW Vehicles1 AS SELECT * FROM Vehicles LIMIT 10;
CREATE VIEW Trips1 AS SELECT * FROM Trips LIMIT 100;
```

1.3 Loading the Data in Partitioned Tables

PostgreSQL provides partitioning mechanisms so that large tables can be split in smaller physical tables. This may result in increased performance when querying and manipulating large tables. We will split the `Trips` table given in the previous section using list partitioning, where each partitition will contain all the trips that start at a particular date. For doing this, we use the procedure given next for automatically creating the partitions according to a date range.

```
CREATE OR REPLACE FUNCTION create_partitions_by_date(TableName TEXT, StartDate DATE,
  EndDate DATE)
RETURNS void AS $$
DECLARE
  d DATE;
  PartitionName TEXT;
BEGIN
  IF NOT EXISTS (
    SELECT 1
    FROM information_schema.tables
    WHERE table_name = lower(TableName))
  THEN
    RAISE EXCEPTION 'Table % does not exist', TableName;
```

```
  END IF;
  IF StartDate >= EndDate THEN
    RAISE EXCEPTION 'The start date % must be before the end date %', StartDate, EndDate;
  END IF;
  d = StartDate;
  WHILE d <= EndDate
  LOOP
    PartitionName = TableName || '_' || to_char(d, 'YYYY_MM_DD');
    IF NOT EXISTS (
      SELECT 1
      FROM information_schema.tables
      WHERE  table_name = lower(PartitionName))
    THEN
      EXECUTE format('CREATE TABLE %s PARTITION OF %s FOR VALUES IN (''%s'');',
        PartitionName, TableName, to_char(d, 'YYYY-MM-DD'));
      RAISE NOTICE 'Partition % has been created', PartitionName;
    END IF;
    d = d + '1 day'::interval;
  END LOOP;
  RETURN;
END
$$ LANGUAGE plpgsql;
```

In order to partition table `Trips` by date we need to add an addition column `TripDate` to table `TripsInput`.

```
ALTER TABLE TripsInput ADD COLUMN TripDate DATE;
UPDATE TripsInput t1
SET TripDate = t2.TripDate
FROM (SELECT DISTINCT TripId, date_trunc('day', MIN(t) OVER (PARTITION BY TripId))
AS TripDate FROM TripsInput) t2
WHERE t1.TripId = t2.TripId;
```

Notice that the `UPDATE` statement above takes into account the fact that a trip may finish at a day later than the starting day.

The following statements create table `Trips` partitioned by date and the associated partitions.

```
DROP TABLE Trips CASCADE;
CREATE TABLE Trips (
  TripId integer,
  TripDate date,
  VehicleId integer NOT NULL REFERENCES Vehicles(VehicleId),
  Trip tgeompoint NOT NULL,
  Trajectory geometry,
  PRIMARY KEY (TripId, TripDate)
) PARTITION BY LIST(TripDate);

SELECT create_partitions_by_date('Trips', (SELECT MIN(TripDate) FROM TripsInput),
(SELECT MAX(TripDate) FROM TripsInput));
```

To see the partitions that have been created automatically we can use the following statement.

```
SELECT I.inhrelid::regclass AS child
FROM pg_inherits I
WHERE i.inhparent = 'trips'::regclass;
```

In our case this would result in the following output.

```
trips_2020_06_01
trips_2020_06_02
trips_2020_06_03
trips_2020_06_04
trips_2020_06_05
```

We modify the query that loads table `Trips` from the data in table `TripsInput` as follows.

```
INSERT INTO Trips
SELECT TripId, TripDate, VehicleId, tgeompoint_seq(array_agg(tgeompoint_inst(
  ST_Transform(ST_SetSRID(ST_MakePoint(PosX,PosY), 4326), 5676), t) ORDER BY t))
FROM TripsInput
GROUP BY TripId, TripDate, VehicleId;
```

We can see how many trips are in each partition of the `TripsInput` as follows.

```
SELECT COUNT(*) FROM trips_2020_06_01;
-- 423
SELECT COUNT(*) FROM trips_2020_06_02;
-- 411
SELECT COUNT(*) FROM trips_2020_06_03;
-- 415
SELECT COUNT(*) FROM trips_2020_06_04;
-- 419
SELECT COUNT(*) FROM trips_2020_06_05;
-- 4
```

Then, we can define the indexes and the views on the table `Trips` as shown in the previous section.

An important advantange of the partitioning mechanism in PostgreSQL is that the constraints and the indexes defined on the `Trips` table are propagated to the partitions as shown next.

```
INSERT INTO Trips VALUES (1, '2020-06-01', 10,
  '[POINT(2389629.8979609837 5626986.483650829)@2020-06-02 08:00]');
-- ERROR:  duplicate key value violates unique constraint "trips_2020_06_01_pkey"
-- DETAIL:  Key (tripid, tripdate)=(1, 2020-06-01) already exists.
```

Similarly, queries on the `Trips` table are propagated to the partitions as shown next.

```
EXPLAIN SELECT COUNT(*) FROM Trips WHERE Trip && tstzspan '[2020-06-02, 2020-06-03)';
```

If there is no index defined on the `Trip` column, the execution plan of the query is as follows:

```
Aggregate (cost=63.64..63.65 rows=1 width=8)
-> Append (cost=0.00..63.62 rows=5 width=0)
   -> Seq Scan on trips_2020_06_01 trips_1  (cost=0.00..11.29 rows=1 width=0)
        Filter: (trip && '[2020-06-02 00:00:00+02, 2020-06-03 00:00:00+02)'::tstzspan)
   -> Seq Scan on trips_2020_06_02 trips_2  (cost=0.00..11.14 rows=1 width=0)
        Filter: (trip && '[2020-06-02 00:00:00+02, 2020-06-03 00:00:00+02)'::tstzspan)
   -> Seq Scan on trips_2020_06_03 trips_3  (cost=0.00..11.19 rows=1 width=0)
        Filter: (trip && '[2020-06-02 00:00:00+02, 2020-06-03 00:00:00+02)'::tstzspan)
   -> Seq Scan on trips_2020_06_04 trips_4  (cost=0.00..10.24 rows=1 width=0)
        Filter: (trip && '[2020-06-02 00:00:00+02, 2020-06-03 00:00:00+02)'::tstzspan)
   -> Seq Scan on trips_2020_06_05 trips_5  (cost=0.00..19.75 rows=1 width=0)
        Filter: (trip && '[2020-06-02 00:00:00+02, 2020-06-03 00:00:00+02)'::tstzspan)
```

After defining an index on the `Trip` column as follows

```
CREATE INDEX Trips_Trip_gist_Idx ON Trips USING gist (Trip);
```

the execution plan of the query is as follows

```
Aggregate (cost=33.73..33.74 rows=1 width=8)
-> Append (cost=0.14..33.71 rows=5 width=0)
   -> Index Scan using trips_2020_06_01_trip_idx on trips_2020_06_01 trips_1
      (cost=0.14..8.16 rows=1 width=0)
        Index Cond: (trip && '[2020-06-02 00:00:00+02, 2020-06-03 00:00:00+02)'::tstzspan)
   -> Index Scan using trips_2020_06_02_trip_idx on trips_2020_06_02 trips_2
```

```
      (cost=0.14..8.16 rows=1 width=0)
        Index Cond: (trip && '[2020-06-02 00:00:00+02, 2020-06-03 00:00:00+02)'::tstzspan)
   -> Index Scan using trips_2020_06_03_trip_idx on trips_2020_06_03 trips_3
      (cost=0.14..8.16 rows=1 width=0)
        Index Cond: (trip && '[2020-06-02 00:00:00+02, 2020-06-03 00:00:00+02)'::tstzspan)
   -> Index Scan using trips_2020_06_04_trip_idx on trips_2020_06_04 trips_4
      (cost=0.14..8.16 rows=1 width=0)
        Index Cond: (trip && '[2020-06-02 00:00:00+02, 2020-06-03 00:00:00+02)'::tstzspan)
   -> Seq Scan on trips_2020_06_05 trips_5  (cost=0.00..1.05 rows=1 width=0)
        Filter: (trip && '[2020-06-02 00:00:00+02, 2020-06-03 00:00:00+02)'::tstzspan)
```

1.4 Exploring the Data

In order to visualize the data with traditional tools such as QGIS we add to table `Trip` a column `Trajectory` of type `geometry` containing the trajectory of the trips.

```
ALTER TABLE Trips ADD COLUMN Trajectory geometry;
UPDATE Trips
SET Trajectory = trajectory(Trip);
```

The visualization of the trajectories in QGIS is given in Figure 1.2. As we will explain in Chapter 2 this synthetic dataset models people using their car for going from home to work in the morning, from work to home in the afternoon, as well as doing some additional leisure trips at evenings or weekends.

In order to know the total number of trips as well as the number of trips we can issue the following queries.

```
SELECT count(*) FROM Trips;
-- 1672
SELECT count(*) FROM Trips WHERE GeometryType(Trajectory) = 'POINT';
-- 0
SELECT count(*) FROM Trips WHERE GeometryType(Trajectory) = 'LINESTRING';
-- 1672
```

We can also determine the spatiotemporal extent of the data using the following query.

```
SELECT extent(Trip) from Trips;
/* SRID=3857;STBOX XT(((469715.0960907607,6577078.768286072),
    (500997.56505993055,6607214.0038881665)),
    [2020-06-01 08:01:16.984+02, 2020-06-05 01:40:04.281127+02]) */
```

We continue investigating the data set by computing the maximum number of concurrent trips over the whole period

```
SELECT maxValue(tcount(Trip)) FROM Trips;
-- 51
```

the average sampling rate

```
SELECT AVG(duration(Trip)/numInstants(Trip)) FROM Trips;
-- 00:00:01.370537
```

and the total travelled distance in kilometers of all trips:

```
SELECT SUM(length(Trip)) / 1e3 as TotalLengthKm FROM Trips;
-- 24209.259034796323
```

Now we want to know the average duration of a trip.

```
SELECT AVG(duration(Trip)) FROM Trips;
-- 00:25:09.065361
```

Figure 1.2: Visualization of the trips in QGIS. The streets are shown in blue, the trips are shown in black, the home nodes in black and the work nodes in red.

This following query tells us the length in kilometers and the duration of each trip.

```
SELECT tripId, length(Trip) / 1e3 AS lengthKm, duration(Trip) AS duration
FROM Trips ORDER BY duration;
```

The following query produces a histogram of trip length.

```
WITH Bins(BinNo, BinRange) AS (
  SELECT 1, floatspan '[0, 1)' UNION
  SELECT 2, floatspan '[1, 2)' UNION
  SELECT 3, floatspan '[2, 5)' UNION
  SELECT 4, floatspan '[5, 10)' UNION
  SELECT 5, floatspan '[10, 50)' UNION
  SELECT 6, floatspan '[50, 100)' ),
Histogram AS (
  SELECT BinNo, BinRange, COUNT(TripId) AS Freq
  FROM Bins LEFT OUTER JOIN Trips ON length(trip) / 1e3 <@ BinRange
  GROUP BY BinNo, BinRange
  ORDER BY BinNo, BinRange )
SELECT BinNo, BinRange, Freq,
  repeat('■', ( Freq::float / MAX(Freq) OVER () * 30 )::int ) AS bar
FROM Histogram;
```

The result of the above query is given next.

```
 BinNo | BinRange   | Freq |              bar
-------+------------+------+--------------------------------
     1 | [0, 1)     |   41 | ■
     2 | [1, 2)     |   91 | ■■■
     3 | [2, 5)     |  329 | ■■■■■■■■■■■
     4 | [5, 10)    |  294 | ■■■■■■■■■■
     5 | [10, 50)   |  909 | ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
     6 | [50, 100)  |    8 |
```

1.5 Querying the Data

We discuss next four categories of queries: range queries, distance queries, temporal aggregate queries, and nearest-neighbor queries.

1.5.1 Range Queries

The queries in this category restrict Trips with respect to a spatial, temporal, or spatio-temporal point or range. In the examples, the spatial points and ranges are given, respectively, in tables `Points` and `Regions`, while temporal points and ranges are given, respectively, in tables `Instants` and `Periods`.

1. List the vehicles that have passed at a region from `Regions`.

   ```
   SELECT DISTINCT r.RegionId, t.VehicleId
   FROM Trips t, Regions r
   WHERE ST_Intersects(trajectory(t.Trip), r.Geom)
   ORDER BY r.RegionId, t.VehicleId;
   ```

   This is a spatial range query. The query verifies that the trajectory of the vehicle intersects the region. PostGIS performs an implicit bounding box comparison `trajectory(t.Trip) && r.Geom` using the spatial index on table `Regions` when executing the predicate `ST_Intersects`.

2. List the vehicles that were within a region from `Regions` during a period from `Periods`.

```
SELECT r.RegionId, p.PeriodId, t.VehicleId
FROM Trips t, Regions r, Periods p
WHERE t.Trip && stbox(r.Geom, p.Period) AND
  eIntersects(atTime(t.Trip, p.Period), r.Geom)
ORDER BY r.RegionId, p.PeriodId, t.VehicleId;
```

This is a spatio-temporal range query. The query performs a bounding box comparison with the `&&` operator using the spatio-temporal index on table `Trips`. After that, the query verifies that the location of the vehicle during the period intersects the region. Notice the predicate `eIntersects` (ever intersects) that is applied to the arguments `atTime(Trip, p.Period)` and `r.Geom`.

3. List the pairs of vehicles that were both located within a region from `Regions` during a period from `Periods`.

```
SELECT DISTINCT t1.VehicleId AS VehId1, t2.VehicleId AS VehId2, r.RegionId, p.PeriodId
FROM Trips t1, Trips t2, Regions r, Periods p
WHERE t1.VehicleId < t2.VehicleId AND t1.Trip && stbox(r.Geom, p.Period) AND
  t2.Trip && stbox(r.Geom, p.Period) AND
  eIntersects(atTime(t1.Trip, p.Period), r.Geom) AND
  eIntersects(atTime(t2.Trip, p.Period), r.Geom)
ORDER BY t1.VehicleId, t2.VehicleId, r.RegionId, p.PeriodId;
```

This is a spatio-temporal range join query. The query selects two trips of different vehicles and performs bounding box comparisons of each trip with a region and a period using the spatio-temporal index of the `Trips` table. The query then verifies that both vehicles were located within the region during the period.

4. List the first time at which a vehicle visited a point in `Points`.

```
SELECT t.VehicleId, p.PointId, MIN(startTimestamp(atValues(t.Trip,p.Geom))) AS Instant
FROM Trips t, Points p
WHERE ST_Contains(trajectory(t.Trip), p.Geom)
GROUP BY t.VehicleId, p.PointId;
```

The query selects a trip and a point and verifies that the vehicle passed by the point by testing that the trajectory of the trip contains the point. Notice that PostGIS will perform the bounding box containment `trajectory(t.Trip) ~ p.Geom` using the spatial index on table `Points` before executing `ST_Contains`. Then, the query projects the trip to the point with the `atValue` function, get the first timestamp of the projected trip with the `startTimestamp` function, and applies the traditional `MIN` aggregate function for all trips of the vehicle and the point.

1.5.2 Temporal Aggregate Queries

There are three common types of temporal aggregate queries.

- Instant temporal aggregate queries in which, from a conceptual perspective, the traditional aggregate function is applied at each instant.
- Window temporal aggregate queries (also known as cumulative queries), which, given a time interval w, compute the value of the aggregate at a time instant t from the values during the time period [t-w, t].
- Span temporal aggregate queries, which, first, split the time line into predefined intervals independently of the target data, and then, for each of these intervals, aggregate the data that overlap the interval.

5. Compute how many vehicles were active at each period in `Periods`.

```
SELECT p.PeriodId, COUNT(*), tCount(atTime(t.Trip, p.Period))
FROM Trips t, Periods p
WHERE t.Trip && p.Period
GROUP BY p.PeriodId
ORDER BY p.PeriodId;
```

This an instant temporal aggregate query. For each period, the query projects the trips to the given period and applies the temporal count to the projected trips. The condition in the `WHERE` clause is used for filtering the trips with the spatio-temporal index on table `Trips`.

6. For each region in `Regions`, give the window temporal count of trips with a 10-minute interval.

```
SELECT r.RegionId, wCount(atGeometry(t.Trip, r.Geom), interval '10 min')
FROM Trips t, Regions r
WHERE t.Trip && stbox(r.Geom)
GROUP BY r.RegionId
HAVING wCount(atGeometry(t.Trip, r.Geom), interval '10 min') IS NOT NULL
ORDER BY r.RegionId;
```

This is a window temporal aggregate query. Suppose that we are computing pollution levels by region. Since the effect of a vehicle passing at a location lasts some time interval, this is a typical case for window aggregates. For each region, the query computes the spatial projection of the trips to the given region and apply the window temporal count to the projected trips. The condition in the `WHERE` clause is used for filtering the trips with the spatio-temporal index. The condition in the `HAVING` clause is used for removing regions that do not intersect with any trip.

7. Count the number of trips that were active during each hour in May 29, 2007.

```
WITH TimeSplit(Period) AS (
  SELECT span(H, H + interval '1 hour')
  FROM generate_series(timestamptz '2007-05-29 00:00:00',
    timestamptz '2007-05-29 23:00:00', interval '1 hour') AS H )
SELECT Period, COUNT(*)
FROM TimeSplit s, Trips t
WHERE s.Period && t.Trip AND atTime(Trip, Period) IS NOT NULL
GROUP BY s.Period
ORDER BY s.Period;
```

This is a span temporal aggregate query. The query defines the intervals to consider in the `TimeSplit` temporary table. For each of these intervals, the main query applies the traditional count function for counting the trips that overlap the interval.

1.5.3 Distance Queries

The queries in this category deal with either the distance travelled by a single object or the distance between two objects. The complexity of the latter queries depend, on the one hand, on whether the reference objects are static or moving, and on the other, on whether the operation required is either the minimum distance ever or the temporal distance computed at each instant.

8. List the overall traveled distances of the vehicles during the periods from `Periods`.

```
SELECT t.VehicleId, p.PeriodId, p.Period,
  SUM(length(atTime(t.Trip, p.Period))) AS Distance
FROM Trips t, Periods p
WHERE t.Trip && p.Period
GROUP BY t.VehicleId, p.PeriodId, p.Period
ORDER BY t.VehicleId, p.PeriodId;
```

The query performs a bounding box comparison with the `&&` operator using the spatio-temporal index on the `Trips` table. It then projects the trip to the period, computes the length of the projected trip, and sum the lengths of all the trips of the same vehicle during the period.

9. List the minimum distance ever between each vehicle and each point from `Points`.

```
SELECT t.VehicleId, p.PointId, MIN(trajectory(t.Trip) <-> p.Geom) AS MinDistance
FROM Trips t, Points p
GROUP BY t.VehicleId, p.PointId
ORDER BY t.VehicleId, p.PointId;
```

The query projects the trip to the spatial dimension with the `trajectory` function and computes the traditional distance between the trajectory of the trip and the point. The traditional minimum function is then applied for computing the minimum distance between all trips of the vehicle and the point.

10. List the minimum temporal distance between each pair of vehicles.

```
SELECT t1.VehicleId AS Veh1Id, t2.VehicleId AS Veh2Id,
  tMin(t1.Trip <-> t2.Trip) AS MinDistance
FROM Trips t1, Trips t2
WHERE t1.VehicleId < t2.VehicleId AND timeSpan(t1.Trip) && timeSpan(t2.Trip)
GROUP BY t1.VehicleId, t2.VehicleId
ORDER BY t1.VehicleId, t2.VehicleId;
```

The query selects two trips `t1` and `t2` from different vehicles that were both traveling during a common period of time, computes the temporal distance between the trips, and then computes the temporal minimum distance between all trips of the two vehicles. The query uses the spatio-temporal index to filter the pairs of trips that were both traveling during a common period of time.

11. List the nearest approach time, distance, and shortest line between each pair of trips.

```
SELECT t1.VehicleId AS Veh1Id, t1.TripId AS Trip1Id, t2.VehicleId AS Veh2Id,
  t2.TripId AS Trip2Id, timeSpan(nearestApproachInstant(t1.Trip, t2.Trip)) AS Time,
  nearestApproachDistance(t1.Trip, t2.Trip) AS Distance,
  shortestLine(t1.Trip, t2.Trip) AS Line
FROM Trips t1, Trips t2
WHERE t1.VehicleId < t2.VehicleId AND timeSpan(t1.Trip) && timeSpan(t2.Trip)
ORDER BY t1.VehicleId, t1.TripId, t2.VehicleId, t2.TripId;
```

This query shows similar functionality as that provided by the PostGIS functions `ST_ClosestPointOfApproach` and `ST_DistanceCPA`. The query selects two trips `t1` and `t2` from different vehicles that were both traveling during a common period of time and computes the required results.

12. List when and where a pairs of vehicles have been at 10 m or less from each other.

```
SELECT t1.VehicleId AS VehId1, t2.VehicleId AS VehId2, atTime(t1.Trip,
  getTime(tDwithin(t1.Trip, t2.Trip, 10.0, TRUE))) AS Position
FROM Trips t1, Trips t2
WHERE t1.VehicleId < t2.VehicleId AND t1.Trip && expandSpace(t2.Trip, 10) AND
  tDwithin(t1.Trip, t2.Trip, 10.0, TRUE) IS NOT NULL
ORDER BY t1.VehicleId, t2.VehicleId, Position;
```

The query performs for each pair of trips `t1` and `t2` of different vehicles a bounding box comparison with the `&&` operator using the spatio-temporal index on the `Trips` table, where the bounding box of `t2` is expanded by 10 m. Then, the `period` expression computes the periods during which the vehicles were within 10 m. from each other and the `atTime` function projects the trips to those periods. Notice that the expression `tDwithin(t1.Trip, t2.Trip, 10.0)` is conceptually equivalent to `dwithin(t1.Trip, t2.Trip) #<= 10.0`. However, in this case the spatio-temporal index cannot be used for filtering values.

1.5.4 Nearest-Neighbor Queries

There are three common types of nearest-neighbor queries in spatial databases.

- k-nearest-neighbor (kNN) queries find the k nearest points to a given point.
- Reverse k-nearest-neighbor (RkNN) queries find the points that have a given point among their k nearest-neighbors.
- Given two sets of points p and Q, aggregate nearest-neighbor (ANN) queries find the points from p that have minimum aggregated distance to all points from Q.

The above types of queries are generalized to temporal points. However, the complexity of these queries depend on whether the reference object and the candidate objects are static or moving. In the examples that follow we only consider the nontemporal version of the nearest-neighbor queries, that is, the one in which the calculation is performed on the projection of temporal points on the spatial dimension. The temporal version of the nearest-neighbor queries remains to be done.

13. For each trip from `Trips`, list the three points from `Points` that have been closest to that vehicle.

```
WITH TripsTraj AS (
  SELECT TripId, VehicleId, trajectory(Trip) AS Trajectory FROM Trips )
SELECT t.VehicleId, ps1.PointId, ps1.Distance
FROM TripsTraj t CROSS JOIN LATERAL (
  SELECT p.PointId, t.Trajectory <-> p.Geom AS Distance
  FROM Points p
  ORDER BY Distance LIMIT 3 ) AS ps1
ORDER BY t.TripId, t.VehicleId, ps1.Distance;
```

This is a nearest-neighbor query with moving reference objects and static candidate objects. The query above uses PostgreSQL's lateral join, which intuitively iterates over each row in a result set and evaluates a subquery using that row as a parameter. The query starts by computing the trajectory of the trips in the temporary table `TripsTraj`. Then, given a trip `t` in the outer query, the subquery computes the traditional distance between the trajectory of `t` and each point `p`. The `ORDER BY` and `LIMIT` clauses in the inner query select the three closest points. PostGIS will use the spatial index on the `Points` table for selecting the three closest points.

14. For each trip from `Trips`, list the three vehicles that are closest to that vehicle

```
SELECT t1.VehicleId AS VehId1, v2.VehicleId AS VehId2, v2.Distance
FROM Trips t1 CROSS JOIN LATERAL (
  SELECT t2.VehicleId, minValue(t1.Trip <-> t2.Trip) AS Distance
  FROM Trips t2
  WHERE t1.VehicleId < t2.VehicleId AND timeSpan(t1.Trip) && timeSpan(t2.Trip)
  ORDER BY Distance LIMIT 3 ) AS v2
ORDER BY t1.VehicleId, v2.VehicleId;
```

This is a nearest-neighbor query where both the reference and the candidate objects are moving. Therefore, it is not possible to proceed as in the previous query to first project the moving points to the spatial dimension and then compute the traditional distance. Given a trip `t1` in the outer query, the subquery computes the temporal distance between `t1` and a trip `t2` of another vehicle different from the vehicle from `t1` and then computes the minimum value in the temporal distance. Finally, the `ORDER BY` and `LIMIT` clauses in the inner query select the three closest vehicles.

15. For each trip from `Trips`, list the points from `Points` that have that vehicle among their three nearest neighbors.

```
WITH TripsTraj AS (
  SELECT TripId, VehicleId, trajectory(Trip) AS Trajectory FROM Trips ),
PointTrips AS (
  SELECT p.PointId, t2.VehicleId, t2.TripId, t2.Distance
  FROM Points p CROSS JOIN LATERAL (
    SELECT t1.VehicleId, t1.TripId, p.Geom <-> t1.Trajectory AS Distance
    FROM TripsTraj t1
    ORDER BY Distance LIMIT 3 ) AS t2 )
SELECT t.VehicleId, t.TripId, p.PointId, pt.Distance
FROM Trips t CROSS JOIN Points p JOIN PointTrips pt
  ON t.VehicleId = pt.VehicleId AND t.TripId = pt.TripId AND p.PointId = pt.PointId
ORDER BY t.VehicleId, t.TripId, p.PointId;
```

This is a reverse nearest-neighbor query with moving reference objects and static candidate objects. The query starts by computing the corresponding nearest-neighbor query in the temporary table `PointTrips` as it is done in Query 13. Then, in the main query it verifies for each trip `t` and point `p` that both belong to the `PointTrips` table.

16. For each trip from `Trips`, list the vehicles having the vehicle of the trip among the three nearest neighbors.

```
WITH TripDistances AS (
  SELECT t1.VehicleId AS VehId1, t1.TripId AS TripId1, t3.VehicleId AS VehId2,
    t3.TripId AS TripId2, t3.Distance
  FROM Trips t1 CROSS JOIN LATERAL (
    SELECT t2.VehicleId, t2.TripId, minValue(t1.Trip <-> t2.Trip) AS Distance
    FROM Trips t2
    WHERE t1.VehicleId < t2.VehicleId AND timeSpan(t1.Trip) && timeSpan(t2.Trip)
    ORDER BY Distance LIMIT 3 ) AS t3 )
SELECT t1.VehicleId, t1.TripId, t2.VehicleId, t2.TripId, td.Distance
FROM Trips t1 JOIN Trips t2 ON t1.VehicleId < t2.VehicleId
  JOIN TripDistances td ON t1.VehicleId = td.VehId1 AND t1.TripId = td.TripId1 AND
  t2.VehicleId = td.VehId2 AND t2.TripId = td.TripId2
ORDER BY t1.VehicleId, t1.TripId, t2.VehicleId, t2.TripId;
```

This is a reverse nearest-neighbor query where both the reference and the candidate objects are moving. The query starts by computing the corresponding nearest-neighbor query in the temporary table `TripDistances` as it is done in Query 14. Then, in the main query it verifies for each pair of trips `t1` and `t2` that both belong to the `TripDistances` table.

17. For each group of ten disjoint vehicles, list the point(s) from `Points`, having the minimum aggregated distance from the given group of ten vehicles during the given period.

```
WITH Groups AS (
  SELECT ((ROW_NUMBER() OVER (ORDER BY v.VehicleId))-1)/10 + 1 AS GroupId, v.VehicleId
  FROM Vehicles v ),
SumDistances AS (
  SELECT g.GroupId, p.PointId,
    SUM(ST_Distance(trajectory(t.Trip), p.Geom)) AS SumDist
  FROM Groups g, Points p, Trips t
  WHERE t.VehicleId = g.VehicleId
  GROUP BY g.GroupId, p.PointId )
SELECT s1.GroupId, s1.PointId, s1.SumDist
FROM SumDistances s1
WHERE s1.SumDist <= ALL (
  SELECT SumDist
  FROM SumDistances s2
  WHERE s1.GroupId = s2.GroupId )
ORDER BY s1.GroupId, s1.PointId;
```

This is an aggregate nearest-neighbor query. The temporary table `Groups` splits the vehicles in groups where the `GroupId` column takes the values from 1 to total number of groups. The temporary table `SumDistances` computes for each group `g` and point `p` the sum of the distances between a trip of a vehicle in the group and the point. The main query then selects for each group in table `SumDistances` the points(s) that have the minimum aggregated distance.

Chapter 2

Generating Realistic Trajectory Datasets

2.1 Introduction

Do you need an arbitrarily large trajectory dataset to tests your ideas? This chapter illustrates how to generate car trips in a city. It implements the BerlinMOD benchmark data generator that is described in:

> Düntgen, C., Behr, T. and Güting, R.H. BerlinMOD: a benchmark for moving object databases. The VLDB Journal 18, 1335 (2009). https://doi.org/10.1007/s00778-009-0142-5

The data generator can be configured by setting the number of simulated cars and the number of simulation days. It models people trips using their cars to and from work during the week as well as some additional leisure trips at evenings or weekends. The simulation uses multiple ideas to be close to reality, including:

- The home locations are sampled with respect to the population statistics of the different administrative areas in the city
- Similarly, the work locations are sampled with respect to employment statistics
- Drivers will try to accelerate to the maximum allowed speed of a road
- Random events will force drivers to slow down or even stop to simulate obstacles, traffic lights, etc.
- Drivers will slow down in curves
- Trips between home and work do not include additional destinations
- Leisure trips start and end at home locations and include multiple destinations

The generator is written in PL/pgSQL, so that it will be easy to insert or adapt simulation rules to reflect other scenarios. It uses MobilityDB types and operations. The generated trajectories are also MobilityDB types. It is controlled by a single parameter, *scale factor*, that determines the size of the generated dataset. Additionally, many other parameters can be used to fine-tune the generation process to reflect various real-world simulation scenarios.

2.2 Contents

This chapter covers the following topics:

- A quick start using the generator
- Understanding the generation process
- Exploring the generated data

- Customizing the generator to your city
- Tuning the generator parameters
- Modifying the generator by changing the simulation scenario
- Creating a network topology from your own streets layer, to be used for the generator

2.3 Tools and Data

- MobilityDB, hence PostgreSQL and PostGIS. The installation instructions can be found here.
- MobilityDB-BerlinMOD. Get the generator from Github here.
- pgRouting. The installation instructions can be found here. The minimum version required is 3.1.0.
- Download the OSM files for Brussels here. Extract the archive in any folder. In the following we refer to this folder as `generatorHome`.

2.4 Quick Start

Running the generator is done in three steps:

Firstly, load the street network. Create a new database `brussels`, then add the extensions hstore, PostGIS, MobilityDB, and pgRouting to it.

```
# in a console:
createdb -h localhost -p 5432 -U dbowner brussels
# replace localhost with your database host, 5432 with your port,
# and dbowner with your database user

psql -h localhost -p 5432 -U dbowner -d brussels -c 'CREATE EXTENSION hstore'
# adds the hstore extension needed by osm2pgsql

psql -h localhost -p 5432 -U dbowner -d brussels -c 'CREATE EXTENSION MobilityDB CASCADE'
# adds the PostGIS and the MobilityDB extensions to the database

psql -h localhost -p 5432 -U dbowner -d brussels -c 'CREATE EXTENSION pgRouting'
# adds the pgRouting extension
```

For the moment, we will use the OSM map of Brussels. It is given in the data section of this workshop in the two files: `brussels.osm, mapconfig.xml`. In the next sections, we will explain how to use other maps. It has been downloaded using the Overpass API, hence it is by default in Spherical Mercator (SRID 3857), which is good for calculating distances. Next load the map and convert it into a routable network topology format suitable for pgRouting.

```
# in a console, go to the generatorHome then:
osm2pgrouting -h localhost -p 5432 -U dbowner -W passwd -f brussels.osm --dbname brussels \
-c mapconfig.xml
```

The configuration file `mapconfig.xml` tells osm2pgrouting which are the roads that will be selected to build the road network as well as the speed limits of the different road types. During the conversion, osm2pgrouting transforms the data into WGS84 (SRID 4326), so we will need later to convert it back to SRID 3857.

Secondly, prepare the base data for the simulation. Now, the street network is ready in the database. The simulation scenario requires to sample home and work locations. To make it realistic, we want to load a map of the administrative regions of Brussels (called communes) and feed the simulator with real population and employment statistics in every commune.

Load the administrative regions from the downloaded `brussels.osm` file, then run the `brussels_generatedata.sql` script using your PostgreSQL client, for example:

```
osm2pgsql -c -H localhost -P 5432 -U dbowner -W -d brussels brussels.osm
# loads all layers in the osm file, including the adminstrative regions

psql -h localhost -p 5432 -U dbowner -d brussels -f brussels_preparedata.sql
# samples home and work nodes, transforms data to SRID 3857, does further data preparation

psql -h localhost -p 5432 -U dbowner -d brussels -f berlinmod_datagenerator.sql
# adds the pgplsql functions of the simulation to the database
```

Finally, run the generator.

```
psql -h localhost -p 5432 -U dbowner -d brussels \
-c 'select berlinmod_generate(scaleFactor := 0.005)'
# calls the main pgplsql function to start the simulation
```

If everything is correct, you should see an output like that starts with this:

```
INFO:  ------------------------------------------------------------------
INFO:  Starting the BerlinMOD data generator with scale factor 0.005
INFO:  ------------------------------------------------------------------
INFO:  Parameters:
INFO:  ------------
INFO:  No. of vehicles = 141, No. of days = 4, Start day = 2020-06-01
INFO:  Path mode = Fastest Path, Disturb data = f
INFO:  Verbosity = minimal, Trip generation = C
...
```

The generator will take about one minute. It will generate trajectories, according to the default parameters, for 141 cars over 4 days starting from Monday, June 1st 2020. As you may have guessed, it is possible to generate more or less data by respectively passing a bigger or a smaller scale factor value. If you want to save the messages produced by the generator in a file you can use a command such as the following one.

```
psql -h localhost -p 5432 -U dbowner -d brussels -c \
"SELECT berlinmod_generate(scaleFactor := 0.005, messages := 'medium')" 2>&1 | \
tee trace.txt
```

You can show more messages describing the generation process by setting the optional parameter `messages` with one of the values `minimal` (the default), `medium`, `verbose`, or `debug`. In Section 2.8 are explained all the parameters that can be used to customize the simulation.

We have shown in Figure 1.2 a visualization of the trips generated in QGIS.

2.5 Exporting the Generated Data

The generated data can be exported, for example, in CSV format using the following queries.

```
COPY (SELECT InstantId, Instant FROM Instants ORDER BY InstantId)
  TO '/home/mobilitydb/data/instants.csv' CSV HEADER DELIMITER ',';
COPY (SELECT LicenceId, Licence, VehicleId FROM Licences ORDER BY LicenceId)
  TO '/home/mobilitydb/data/licences.csv' CSV HEADER DELIMITER ',';
COPY (SELECT PeriodId, lower(Period) AS StartP, upper(Period) AS EndP FROM Periods
    ORDER BY PeriodId)
  TO '/home/mobilitydb/data/periods.csv' CSV HEADER DELIMITER ',';
COPY (SELECT PointId, ST_X(Geom) AS PosX, ST_Y(Geom) AS PosY FROM Points ORDER BY PointId)
  TO '/home/mobilitydb/data/points.csv' CSV HEADER DELIMITER ',';
COPY (
  SELECT RegionId, (dp).Path[2] AS PointID, ST_X((dp).Geom) AS PosX,
    ST_Y((dp).Geom) AS PosY
  FROM (SELECT RegionId, ST_DumpPoints(ST_Transform(Geom, 4326)) AS dp FROM Regions) AS t
```

```
  ) TO '/home/mobilitydb/data/regions.csv' CSV HEADER DELIMITER ',';
COPY (
  WITH Temp1 AS (
    SELECT TripId, VehicleId, unnest(instants(Trip)) AS Inst FROM Trips ),
  Temp2 AS (
    SELECT TripId, VehicleId, ST_Transform(getValue(Inst),4326) AS Point,
      getTimestamp(Inst) AS t FROM Temp1 )
  SELECT TripId, VehicleId, ST_X(Point) AS PosX, ST_Y(Point) AS PosY, t
  FROM Temp2
  ORDER BY TripId, VehicleId, t
  ) TO '/home/mobilitydb/data/trips.csv' CSV HEADER DELIMITER ',';
COPY (SELECT VehicleId, Licence, VehicleType, Model FROM SELECT Vehicles
    ORDER BY VehicleId)
  TO '/home/mobilitydb/data/vehicles.csv' CSV HEADER DELIMITER ',';
```

Actually, the data we used in Chapter 1 was exported by running the BerlinMOD generator with OSM data for Brussels with the scale factor 0.005.

2.6 Exploring the Generated Data

Now use a PostgreSQL client such as psql or pgAdmin to explore the properties of the generated trajectories. We start by obtaining some statistics about the number, the total duration, and the total length in Km of the trips.

```
SELECT COUNT(*), SUM(duration(Trip)), SUM(length(Trip)) / 1e3
FROM Trips;
-- 1686  "618:34:23.478239"  20546.31859281626
```

We continue by further analyzing the duration of all the trips

```
SELECT MIN(duration(Trip)), MAX(duration(Trip)), AVG(duration(Trip))
FROM Trips;
-- "00:00:29.091033"  "01:13:21.225514"  "00:22:02.365486"
```

or the duration of the trips by trip type.

```
SELECT
  CASE
    WHEN t.SourceNode = v.Home AND date_part('dow', t.day) BETWEEN 1 AND 5 AND
      date_part('hour', startTimestamp(Trip)) < 12 THEN 'home_work'
    WHEN t.SourceNode = v.Work AND date_part('dow', t.day) BETWEEN 1 AND 5 AND
      date_part('hour', startTimestamp(Trip)) > 12  THEN 'work_home'
    WHEN date_part('dow', t.day) BETWEEN 1 AND 5 THEN 'leisure_weekday'
    ELSE 'leisure_weekend'
  END AS TripType, COUNT(*), MIN(duration(Trip)), MAX(duration(Trip)), AVG(duration(Trip))
FROM Trips t, Vehicles v
WHERE t.VehicleId = v.VehicleId
GROUP BY TripType;
-- "leisure_weekday"    558    "00:00:29.091033"  "00:57:30.195709"  "00:10:59.118318"
-- "work_home"          564    "00:02:04.159342"  "01:13:21.225514"  "00:27:33.424924"
-- "home_work"          564    "00:01:57.456419"  "01:11:44.551344"  "00:27:25.145454"
```

As can be seen, no weekend leisure trips have been generated, which is normal since the data generated covers four days starting on Monday, June 1st 2020.

We can analyze further the length in Km of the trips as follows.

```
SELECT MIN(length(Trip)) / 1e3, MAX(length(Trip)) / 1e3, AVG(length(Trip)) / 1e3
FROM Trips;
-- 0.2731400585134866  53.76566616928331  12.200901777206806
```

As can be seen the longest trip is more than 56 Km long. Let's visualize one of these long trips.

```
SELECT VehicleId, SeqNo, source, target, round(length(Trip)::numeric / 1e3, 3),
  startTimestamp(Trip), duration(Trip)
FROM Trips
WHERE length(Trip) > 50000 LIMIT 1;
-- 90  1  23078  11985  53.766  "2020-06-01 08:46:55.487+02"  "01:10:10.549413"
```

We can then visualize this trip in PostGIS. As can be seen, in Figure 2.1, the home and the work nodes of the vehicle are located at two extremities in Brussels.

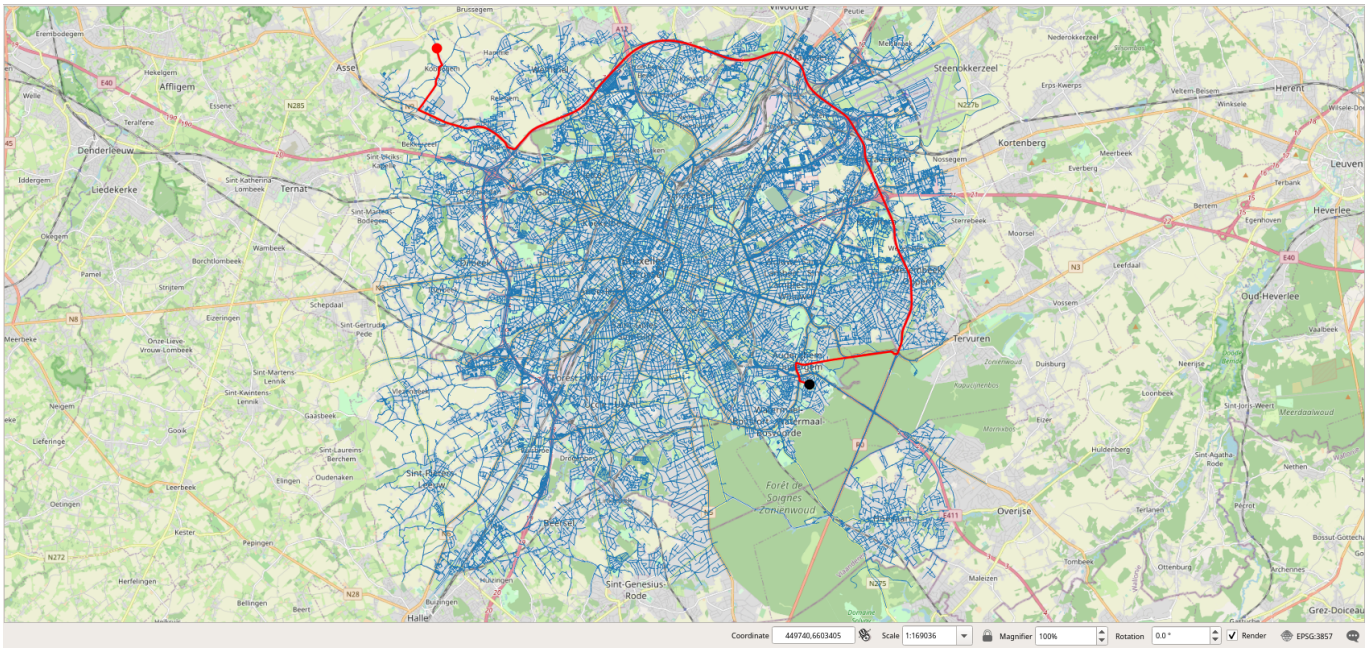

Figure 2.1: Visualization of a long trip.

We can obtain some statistics about the average speed in Km/h of all the trips as follows.

```
SELECT MIN(twAvg(speed(Trip))) * 3.6, MAX(twAvg(speed(Trip))) * 3.6,
  AVG(twAvg(speed(Trip))) * 3.6
FROM Trips;
-- 14.211962789552468  53.31779380411017  31.32438581663778
```

A possible visualization that we could envision is to use gradients to show how the edges of the network are used by the trips. We start by determining how many trips traversed each of the edges of the network as follows.

```
CREATE TABLE HeatMap AS
SELECT e.id, e.Geom, COUNT(*)
FROM Edges e, Trips t
WHERE ST_Intersects(e.Geom, t.trajectory)
GROUP BY e.id, e.Geom;
```

This is an expensive query since it took 42 min in my laptop. In order to display unused edges in our visualization we need to add them to the table with a count of 0.

```
INSERT INTO HeatMap
SELECT e.id, e.Geom, 0 FROM Edges e WHERE e.id NOT IN (
  SELECT id FROM HeatMap );
```

We need some basic statistics about the attribute `COUNT` in order to define the gradients.

```
SELECT MIN(count), MAX(COUNT), round(AVG(COUNT), 3), round(STDDEV(COUNT), 3) FROM HeatMap;
-- 0 204 4.856 12.994
```

Although the maximum value is 204, the average and the standard deviation are, respectively, around 5 and 13.

In order to display in QGIS the edges of the network with a gradient according to the attribute `count`, we use the following expression.

```
ramp_color('RdGy', scale_linear(count, 0, 10, 0, 1))
```

The `scale_linear` function transforms the value of the attribute `count` into a value in [0,1], as stated by the last two parameters. As stated by the two other parameters 0 and 10, which define the range of values to transform, we decided to assign a full red color to an edge as soon as there are at least 10 trips that traverse the edge. The `ramp_color` function states the gradient to be used for the display, in our case from blue to red. The usage of this expression in QGIS is shown in Figure 2.2 and the resulting visualization is shown in Figure 2.3.

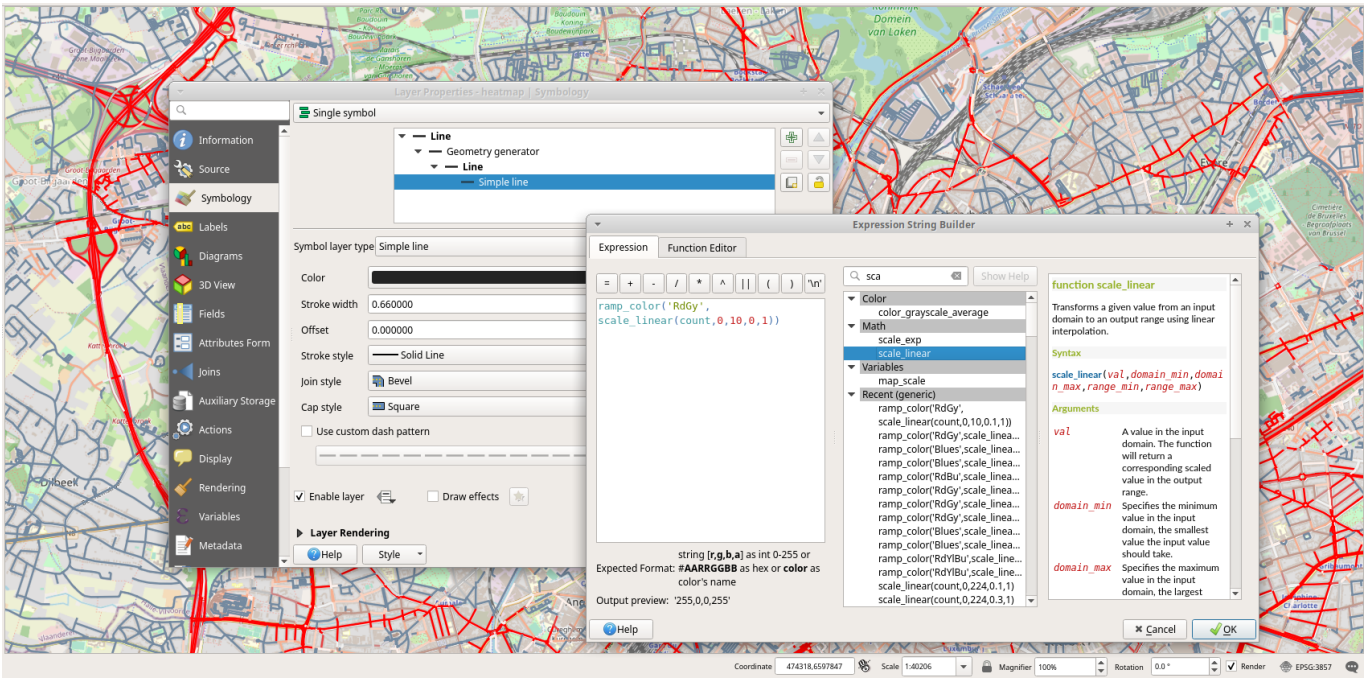

Figure 2.2: Assigning in QGIS a gradient color from blue to red according to the value of the attribute `count`.

Another possible visualization is to use gradients to show the speed used by the trips to traverse the edges of the network. As the maximum speed of edges varies from 20 to 120 Km/h, what would be interesting to compare is the speed of the trips at an edge with respect to the maximum speed of the edge. For this we issue the following query.

```
DROP TABLE IF EXISTS EdgeSpeed;
CREATE TABLE EdgeSpeed AS
SELECT p.edge, twAvg(speed(atGeometry(t.Trip, ST_Buffer(p.Geom, 0.1)))) * 3.6 AS twAvg
FROM Trips t, Paths p
WHERE t.source = p.start_vid AND t.target = p.end_vid AND p.edge > 0
ORDER BY p.edge;
```

This is an even more expensive query than the previous one since it took more than 2 hours in my laptop. Given a trip and an edge, the query restricts the trip to the geometry of the edge and computes the time-weighted average of the speed. Notice that the `ST_Buffer` is used to cope with the floating-point precision. After that we can compute the speed map as follows.

```
CREATE TABLE SpeedMap AS
WITH Temp AS (
```

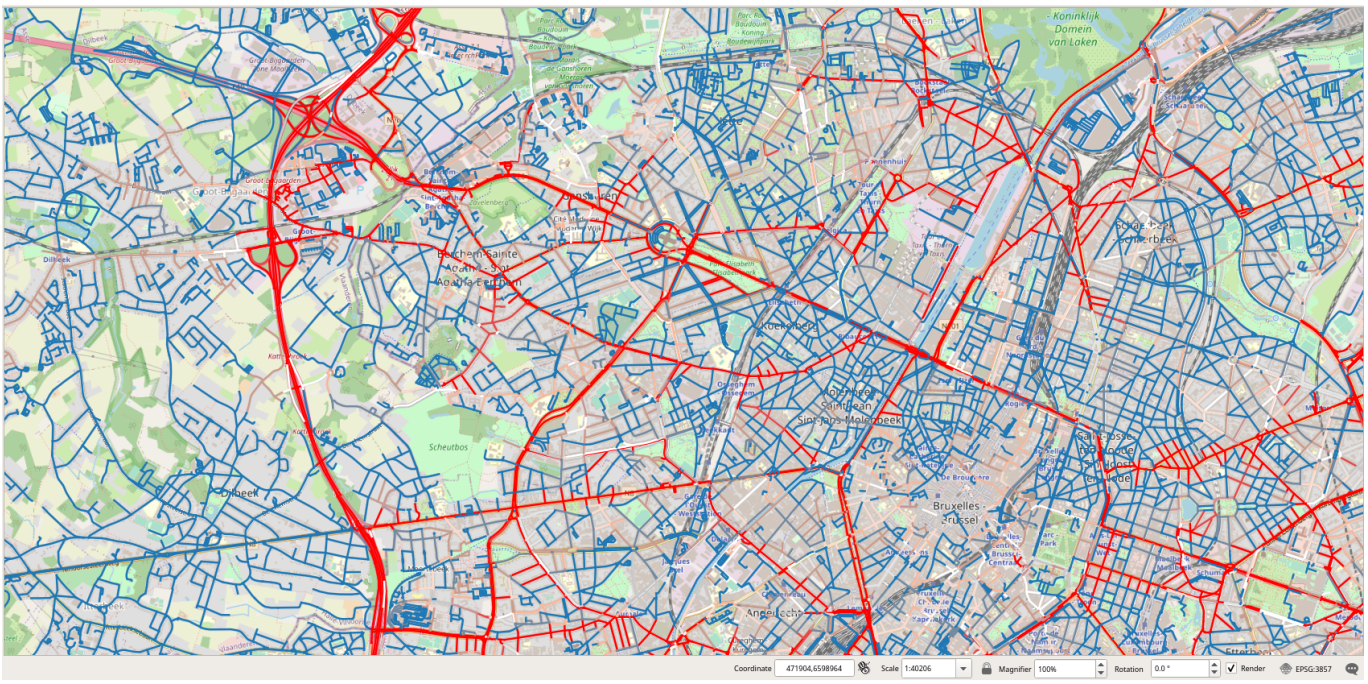

Figure 2.3: Visualization of the edges of the graph according to the number of trips that traversed the edges.

```
  SELECT edge, avg(twAvg) FROM EdgeSpeed GROUP BY edge )
SELECT id, maxspeed_forward AS maxspeed, Geom, avg, avg / maxspeed_forward AS perc
FROM Edges e, Temp t
WHERE e.id = t.edge;
```

Figure 2.4 shows the visualization of the speed map without and with the base map.

2.7 Understanding the Generation Process

We describe next the main steps in the generation of the BerlinMOD scenario. The generator uses multiple parameters that can be set to customize the generation process. We explain in detail these parameters in Section 2.9. It is worth noting that the procedures explained in this section have been slightly simplified with respect to the actual procedures by removing ancillary details concerning the generation of tracing messages at various verbosity levels.

We start by creating a first set of tables for containing the generated data as follows.

```
CREATE TABLE VehicleNodes(VehicleId int PRIMARY KEY, HomeNode bigint NOT NULL,
  WorkNode bigint NOT NULL, NoNeighbours int);
CREATE TABLE Vehicles(VehicleId int PRIMARY KEY, Licence text,
  VehicleType text, Model text);
CREATE TABLE Destinations(VehicleId int, SourceNode bigint, TargetNode bigint,
  PRIMARY KEY (VehicleId, SourceNode, TargetNode));
CREATE TABLE Licences(VehicleId int PRIMARY KEY, Licence text, VehicleType text);
CREATE TABLE Neighbourhoods(VehicleId int, SeqNo int, Node bigint NOT NULL,
  PRIMARY KEY (VehicleId, SeqNo));

-- Get the number of nodes
SELECT COUNT(*) INTO noNodes FROM Nodes;

FOR vehId IN 1..noVehicles LOOP
-- Fill the Vehicles table
IF nodeChoice = 'Network Based' THEN
```

Figure 2.4: Visualization of the edges of the graph according to the speed of trips that traversed the edges.

```
  homeNode = random_int(1, noNodes);
  workNode = random_int(1, noNodes);
ELSE
  homeNode = berlinmod_selectHomeNode();
  workNode = berlinmod_selectWorkNode();
END IF;
IF homeNode IS NULL OR workNode IS NULL THEN
  RAISE EXCEPTION '    The home and the work nodes cannot be NULL';
END IF;
INSERT INTO Vehicles VALUES (vehId, homeNode, workNode);

-- Fill the Destinations table
INSERT INTO Destinations(VehicleId, SourceNode, TargetNode) VALUES
  (vehId, homeNode, workNode), (vehId, workNode, homeNode);

-- Fill the Licences table
licence = berlinmod_createLicence(vehId);
type = berlinmod_vehicleType();
model = berlinmod_vehicleModel();
INSERT INTO Licences VALUES (vehId, licence, type, model);

-- Fill the Neighbourhoods table
INSERT INTO Neighbourhoods
WITH Temp AS (
  SELECT vehId AS VehicleId, n2.NodeId AS Node
  FROM Nodes n1, Nodes n2
  WHERE n1.NodeId = homeNode AND n1.NodeId <> n2.NodeId AND
    ST_DWithin(n1.Geom, n2.Geom, P_NEIGHBOURHOOD_RADIUS) )
SELECT vehId, ROW_NUMBER() OVER () AS SeqNo, Node
FROM Temp;
END LOOP;

CREATE UNIQUE INDEX Vehicles_VehicleId_idx ON Vehicles USING BTREE(VehicleId);
CREATE UNIQUE INDEX Neighbourhoods_pkey_idx ON Neighbourhoods USING BTREE(VehicleId, SeqNo) ↩
    ;

UPDATE Vehicles v
SET NoNeighbours = (SELECT COUNT(*) FROM Neighbourhoods n WHERE n.VehicleId = v.VehicleId);
```

We start by storing in the `Vehicles` table the home and the work node of each vehicle. Depending on the value of the variable `nodeChoice`, we chose these nodes either with a uniform distribution among all nodes in the network or we call specific functions that take into account population and employment statistics in the area covered by the generation. We then keep track in the `Destinations` table of the two trips to and from work and we store in the `Licences` table information describing the vehicle. Finally, we compute in the `Neighbourhoods` table the set of nodes that are within a given distance of the home node of every vehicle. This distance is stated by the parameter `P_NEIGHBOURHOOD_RADIUS`, which is set by default to 3 Km.

We create now auxiliary tables containing benchmarking data. The number of rows these tables is determined by the parameter `P_SAMPLE_SIZE`, which is set by default to 100. These tables are used by the BerlinMOD benchmark to assess the performance of various types of queries.

```
CREATE TABLE Points(PointId int PRIMARY KEY, Geom geometry(Point));
INSERT INTO Points
WITH Temp AS (
  SELECT PointId, random_int(1, noNodes) AS Node
  FROM generate_series(1, P_SAMPLE_SIZE) PointId )
SELECT t.PointId, n.Geom
FROM Temp t, Nodes n
WHERE t.Node = n.PointId;

CREATE TABLE Regions(RegionId int PRIMARY KEY, Geom geometry(Polygon));
INSERT INTO Regions
WITH Temp AS (
```

```
  SELECT RegionId, random_int(1, noNodes) AS Node
  FROM generate_series(1, P_SAMPLE_SIZE) RegionId )
SELECT t.RegionId, ST_Buffer(n.Geom, random_int(1, 997) + 3.0, random_int(0, 25)) AS Geom
FROM Temp t, Nodes n
WHERE t.Node = n.RegionId;

CREATE TABLE Instants(InstantId int PRIMARY KEY, Instant timestamptz);
INSERT INTO Instants
SELECT InstantId, startDay + (random() * noDays) * interval '1 day' AS Instant
FROM generate_series(1, P_SAMPLE_SIZE) InstantId;

CREATE TABLE Periods(PeriodId int PRIMARY KEY, period tstzspan);
INSERT INTO Periods
WITH Temp AS (
  SELECT PeriodId, startDay + (random() * noDays) * interval '1 day' AS Instant
  FROM generate_series(1, P_SAMPLE_SIZE) PeriodId )
SELECT PeriodId, span(instant, instant + abs(random_gauss()) * interval '1 day',
  true, true) AS Period
FROM Temp;
```

We generate now the leisure trips. There is at most one leisure trip in the evening of a week day and at most two leisure trips each day of the weekend, one in the morning and another one in the afternoon. Each leisure trip is composed of 1 to 3 destinations. The leisure trip starts and ends at the home node and visits successively these destinations. In our implementation, the various subtrips from a source to a destination node of a leisure trip are encoded independently, contrary to what is done in Secondo where a leisure trip is encoded as a single trip and stops are added between successive destinations.

```
CREATE TABLE LeisureTrip(VehicleId int, StartDate date, TripNo int, SeqNo int,
  SourceNode bigint, TargetNode bigint,
  PRIMARY KEY (VehicleId, StartDate, TripNo, SeqNo));
-- Loop for every vehicle
FOR vehId IN 1..noVehicles LOOP
-- Get home node and number of neighbour nodes
SELECT home, NoNeighbours INTO homeNode, noNeigh
FROM Vehicles v WHERE v.VehicleId = i;
day = startDay;
-- Loop for every generation day
FOR dayNo IN 1..noDays LOOP
  weekday = date_part('dow', day);
  -- Generate leisure trips (if any)
  -- 1: Monday, 5: Friday
  IF weekday BETWEEN 1 AND 5 THEN
    noLeisTrips = 1;
  ELSE
    noLeisTrips = 2;
  END IF;
  -- Loop for every leisure trip in a day (1 or 2)
  FOR leis IN 1..noLeisTrips LOOP
    -- Generate a leisure trip with a 40% probability
    IF random() <= 0.4 THEN
      -- Select a number of destinations between 1 and 3
      IF random() < 0.8 THEN
        noDest = 1;
      ELSIF random() < 0.5 THEN
        noDest = 2;
      ELSE
        noDest = 3;
      END IF;
      sourceN = homeN;
      FOR dest IN 1..noDest + 1 LOOP
        IF dest <= noDest THEN
          targetNode = berlinmod_selectDestNode(i, noNeigh, noNodes);
        ELSE
```

```
          targetNode = homeNode;
        END IF;
        IF targetNode IS NULL THEN
          RAISE EXCEPTION '    Destination node cannot be NULL';
        END IF;
        INSERT INTO LeisureTrip VALUES
          (vehId, day, leis, dest, sourceN, targetN);
        INSERT INTO Destinations(VehicleId, SourceNode, TargetNode) VALUES
          (vehId, sourceNode, targetNode) ON CONFLICT DO NOTHING;
        sourceNode = targetNode;
      END LOOP;
    END IF;
  END LOOP;
  day = day + 1 * interval '1 day';
END LOOP;
END LOOP;

CREATE INDEX Destinations_vehicle_idx ON Destinations USING BTREE(VehicleId);
```

For each vehicle and each day, we determine the number of potential leisure trips depending on whether it is a week or weekend day. A leisure trip is generated with a probability of 40% and is composed of 1 to 3 destinations. These destinations are chosen so that 80% of the destinations are from the neighbourhood of the vehicle and 20% are from the complete graph. The information about the composition of the leisure trips is then added to the `LeisureTrip` and `Destinations` tables.

We then call pgRouting to generate the path for each source and destination nodes in the `Destinations` table.

```
CREATE TABLE Paths(
  -- This attribute is needed for partitioning the table for big scale factors
  vehicle int,
  -- The following attributes are generated by pgRouting
  start_vid bigint, end_vid bigint, seq int, node bigint, edge bigint,
  -- The following attributes are filled from the Edges table
  Geom geometry NOT NULL, speed float NOT NULL, category int NOT NULL,
  PRIMARY KEY (VehicleId, start_vid, end_vid, seq));

-- Select query sent to pgRouting
IF pathMode = 'Fastest Path' THEN
query1_pgr = 'SELECT id, source, target, cost_s AS cost,'
  'reverse_cost_s as reverse_cost FROM edges';
ELSE
query1_pgr = 'SELECT id, source, target, length_m AS cost,'
  'length_m * sign(reverse_cost_s) as reverse_cost FROM edges';
END IF;
-- Get the total number of paths and number of calls to pgRouting
SELECT COUNT(*) INTO noPaths FROM (SELECT DISTINCT source, target FROM Destinations) AS t;
noCalls = ceiling(noPaths / P_PGROUTING_BATCH_SIZE::float);

FOR i IN 1..noCalls LOOP
query2_pgr = format('SELECT DISTINCT source, target FROM Destinations '
  'ORDER BY source, target LIMIT %s OFFSET %s',
  P_PGROUTING_BATCH_SIZE, (i - 1) * P_PGROUTING_BATCH_SIZE);
INSERT INTO Paths(VehicleId, start_vid, end_vid, seq, node, edge, Geom, speed, category)
WITH Temp AS (
  SELECT start_vid, end_vid, path_seq, node, edge
  FROM pgr_dijkstra(query1_pgr, query2_pgr, true)
  WHERE edge > 0 )
SELECT d.VehicleId, start_vid, end_vid, path_seq, node, edge,
  -- adjusting direction of the edge traversed
  CASE
    WHEN t.node = e.source THEN e.Geom
    ELSE ST_Reverse(e.Geom)
  END AS Geom, e.maxspeed_forward AS speed,
```

```
  berlinmod_roadCategory(e.tag_id) AS category
FROM Destinations d, Temp t, Edges e
WHERE d.source = t.start_vid AND d.target = t.end_vid AND e.id = t.edge;
END LOOP;

CREATE INDEX Paths_vehicle_start_vid_end_vid_idx ON Paths USING
BTREE(VehicleId, start_vid, end_vid);
```

The variable `pathMode` determines whether pgRouting computes either the fastest or the shortest path from a source to a destination node. Then, we determine the number of calls to pgRouting. Indeed, depending on the available memory of the computer, there is a limit in the number of paths to be computed by pgRouting in a single call. The paths are stored in the `Paths` table. In addition to the columns generated by pgRouting, we add the geometry (adjusting the direction if necessary), the maximum speed, and the category of the edge. The BerlinMOD data generator considers three road categories: side road, main road, and freeway. The OSM road types are mapped to one of these categories in the function `berlinmod_roadCategory`.

We are now ready to generate the trips.

```
DROP TYPE IF EXISTS step CASCADE;
CREATE TYPE step AS (linestring geometry, maxspeed float, category int);

CREATE FUNCTION berlinmod_createTrips(noVehicles int, noDays int, startDay date,
  disturbData boolean)
RETURNS void LANGUAGE plpgsql STRICT AS $$
DECLARE
  /* Declaration of variables and parameters ... */
BEGIN
  DROP TABLE IF EXISTS Trips;
  CREATE TABLE Trips(VehicleId int, day date, seq int, source bigint, target bigint,
    Trip tgeompoint, trajectory geometry, PRIMARY KEY (VehicleId, day, seq));
  -- Loop for each vehicle
  FOR i IN 1..noVehicles LOOP
    -- Get home -> work and work -> home paths
    SELECT home, work INTO homeNode, workNode
    FROM Vehicles v WHERE v.VehicleId = i;
    SELECT array_agg((Geom, speed, category)::step ORDER BY seq) INTO homework
    FROM Paths WHERE VehicleId = i AND start_vid = homeNode AND end_vid = workNode;
    SELECT array_agg((Geom, speed, category)::step ORDER BY seq) INTO workhome
    FROM Paths WHERE VehicleId = i AND start_vid = workNode AND end_vid = homeNode;
    d = startDay;
    -- Loop for each generation day
    FOR j IN 1..noDays LOOP
      weekday = date_part('dow', d);
      -- 1: Monday, 5: Friday
      IF weekday BETWEEN 1 AND 5 THEN
        -- Crete trips home -> work and work -> home
        t = d + time '08:00:00' + CreatePauseN(120);
        createTrip(homework, t, disturbData);
        INSERT INTO Trips VALUES (i, d, 1, homeNode, workNode, trip, trajectory(trip));
        t = d + time '16:00:00' + CreatePauseN(120);
        trip = createTrip(workhome, t, disturbData);
        INSERT INTO Trips VALUES (i, d, 2, workNode, homeNode, trip, trajectory(trip));
        tripSeq = 2;
      END IF;
      -- Get the number of leisure trips
      SELECT COUNT(DISTINCT tripNo) INTO noLeisTrip
      FROM LeisureTrip L
      WHERE L.VehicleId = i AND L.day = d;
      -- Loop for each leisure trip (0, 1, or 2)
      FOR k IN 1..noLeisTrip LOOP
        IF weekday BETWEEN 1 AND 5 THEN
          t = d + time '20:00:00' + CreatePauseN(90);
          leisNo = 1;
```

```
        ELSE
          -- Determine whether it is a morning/afternoon (1/2) trip
          IF noLeisTrip = 2 THEN
            leisNo = k;
          ELSE
            SELECT tripNo INTO leisNo FROM LeisureTrip L
            WHERE L.VehicleId = i AND L.day = d LIMIT 1;
          END IF;
          -- Determine the start time
          IF leisNo = 1 THEN
            t = d + time '09:00:00' + CreatePauseN(120);
          ELSE
            t = d + time '17:00:00' + CreatePauseN(120);
        END IF;
        END IF;
        -- Get the number of subtrips (number of destinations + 1)
        SELECT count(*) INTO noSubtrips
        FROM LeisureTrip L
        WHERE L.VehicleId = i AND L.tripNo = leisNo AND L.day = d;
        FOR m IN 1..noSubtrips LOOP
          -- Get the source and destination nodes of the subtrip
          SELECT source, target INTO sourceNode, targetNode
          FROM LeisureTrip L
          WHERE L.VehicleId = i AND L.day = d AND L.tripNo = leisNo AND L.seq = m;
          -- Get the path
          SELECT array_agg((Geom, speed, category)::step ORDER BY seq) INTO Path
          FROM Paths p
          WHERE VehicleId = i AND start_vid = sourceNode AND end_vid = targetNode;
          trip = createTrip(Path, t, disturbData);
          tripSeq = tripSeq + 1;
          INSERT INTO Trips VALUES
            (i, d, tripSeq, sourceNode, targetNode, trip, trajectory(trip));
          -- Add a delay time in [0, 120] min using a bounded Gaussian distribution
          t = endTimestamp(trip) + createPause();
        END LOOP;
      END LOOP;
      d = d + 1 * interval '1 day';
    END LOOP;
  END LOOP;
  RETURN;
END; $$
```

We create a type `step` which is a record composed of the geometry, the maximum speed, and the category of an edge. The procedure loops for each vehicle and each day and calls the procedure `createTrip` for creating the trips. If the day is a weekday, we generate the trips from home to work and from work to home starting, respectively, at 8 am and 4 pm plus a random non-zero duration of 120 minutes using a uniform distribution. We then generate the leisure trips. During the week days, the possible evening leisure trip starts at 8 pm plus a random random non-zero duration of 90 minutes, while during the weekend days, the two possible morning and afternoon trips start, respectively, at 9 am and 5 pm plus a random non-zero duration of 120 minutes. Between the multiple destinations of a leisure trip we add a delay time of maximum 120 minutes using a bounded Gaussian distribution.

Finally, we explain the procedure that create a trip.

```
CREATE OR REPLACE FUNCTION createTrip(edges step[], startTime timestamptz,
  disturbData boolean)
RETURNS tgeompoint LANGUAGE plpgsql STRICT AS $$
DECLARE
  /* Declaration of variables and parameters ... */
BEGIN
  srid = ST_SRID((edges[1]).linestring);
  p1 = ST_PointN((edges[1]).linestring, 1); x1 = ST_X(p1); y1 = ST_Y(p1);
  curPos = p1; t = startTime;
```

```
  instants[1] = tgeompoint_inst(p1, t);
  curSpeed = 0; l = 2; noEdges = array_length(edges, 1);
  -- Loop for every edge
  FOR i IN 1..noEdges LOOP
    -- Get the information about the current edge
    linestring = (edges[i]).linestring; maxSpeedEdge = (edges[i]).maxSpeed;
    category = (edges[i]).category;
    -- Determine the number of segments
    SELECT array_agg(Geom ORDER BY Path) INTO points
    FROM ST_DumpPoints(linestring);
    noSegs = array_length(points, 1) - 1;
    -- Loop for every segment
    FOR j IN 1..noSegs LOOP
      p2 = points[j + 1]; x2 = ST_X(p2); y2 = ST_Y(p2);
      -- If there is a segment ahead in the current edge compute the angle of the turn
      -- and the maximum speed at the turn depending on this angle
      IF j < noSegs THEN
        p3 = points[j + 2];
        alpha = degrees(ST_Angle(p1, p2, p3));
        IF abs(mod(alpha::numeric, 360.0)) < P_EPSILON THEN
          maxSpeedTurn = maxSpeedEdge;
        ELSE
          maxSpeedTurn = mod(abs(alpha - 180.0)::numeric, 180.0) / 180.0 * maxSpeedEdge;
        END IF;
      END IF;
      -- Determine the number of fractions
      segLength = ST_Distance(p1, p2);
      IF segLength < P_EPSILON THEN
        RAISE EXCEPTION 'Segment % of edge % has zero length', j, i;
      END IF;
      fraction = P_EVENT_LENGTH / segLength;
      noFracs = ceiling(segLength / P_EVENT_LENGTH);
      -- Loop for every fraction
      k = 1;
      WHILE k <= noFracs LOOP
        -- If the current speed is zero, apply an acceleration event
        IF curSpeed <= P_EPSILON_SPEED THEN
          curSpeed = least(P_EVENT_ACC, maxSpeedEdge);
        ELSE
          -- If the current speed is not zero, apply a deceleration or a stop event
          -- with a probability proportional to the maximun speed
          IF random() <= P_EVENT_C / maxSpeedEdge THEN
            IF random() <= P_EVENT_P THEN
              -- Apply a stop event
              curSpeed = 0.0;
            ELSE
              -- Apply a deceleration event
              curSpeed = curSpeed * random_binomial(20, 0.5) / 20.0;
            END IF;
          ELSE
            -- Otherwise, apply an acceleration event
            IF k = noFracs AND j < noSegs THEN
              maxSpeed = maxSpeedTurn;
            ELSE
              maxSpeed = maxSpeedEdge;
            END IF;
            curSpeed = least(curSpeed + P_EVENT_ACC, maxSpeed);
          END IF;
        END IF;
        -- If speed is zero add a wait time
        IF curSpeed < P_EPSILON_SPEED THEN
          waitTime = random_exp(P_DEST_EXPMU);
```

```
        IF waitTime < P_EPSILON THEN
          waitTime = P_DEST_EXPMU;
        END IF;
        t = t + waitTime * interval '1 sec';
      ELSE
        -- Otherwise, move current position towards the end of the segment
        IF k < noFracs THEN
          x = x1 + ((x2 - x1) * fraction * k);
          y = y1 + ((y2 - y1) * fraction * k);
          IF disturbData THEN
            dx = (2 * P_GPS_STEPMAXERR * rand()) - P_GPS_STEPMAXERR;
            dy = (2 * P_GPS_STEPMAXERR * rand()) - P_GPS_STEPMAXERR;
            errx = errx + dx; erry = erry + dy;
            IF errx > P_GPS_TOTALMAXERR THEN
              errx = P_GPS_TOTALMAXERR;
            END IF;
            IF errx < - 1 * P_GPS_TOTALMAXERR THEN
              errx = -1 * P_GPS_TOTALMAXERR;
            END IF;
            IF erry > P_GPS_TOTALMAXERR THEN
              erry = P_GPS_TOTALMAXERR;
            END IF;
            IF erry < -1 * P_GPS_TOTALMAXERR THEN
              erry = -1 * P_GPS_TOTALMAXERR;
            END IF;
            x = x + dx; y = y + dy;
          END IF;
          curPos = ST_SetSRID(ST_Point(x, y), srid);
          curDist = P_EVENT_LENGTH;
        ELSE
          curPos = p2;
          curDist = segLength - (segLength * fraction * (k - 1));
        END IF;
        travelTime = (curDist / (curSpeed / 3.6));
        IF travelTime < P_EPSILON THEN
          travelTime = P_DEST_EXPMU;
        END IF;
        t = t + travelTime * interval '1 sec';
        k = k + 1;
      END IF;
      instants[l] = tgeompoint_inst(curPos, t);
      l = l + 1;
    END LOOP;
    p1 = p2; x1 = x2; y1 = y2;
  END LOOP;
  -- If we are not already in a stop, apply a stop event with a probability
  -- depending on the category of the current edge and the next one (if any)
  IF curSpeed > P_EPSILON_SPEED AND i < noEdges THEN
    nextCategory = (edges[i + 1]).category;
    IF random() <= P_DEST_STOPPROB[category][nextCategory] THEN
      curSpeed = 0;
      waitTime = random_exp(P_DEST_EXPMU);
      IF waitTime < P_EPSILON THEN
        waitTime = P_DEST_EXPMU;
      END IF;
      t = t + waitTime * interval '1 sec';
      instants[l] = tgeompoint_inst(curPos, t);
      l = l + 1;
    END IF;
  END IF;
END LOOP;
RETURN tgeompoint_seq(instants, true, true, true);
```

```
END; $$
```

The procedure receives as first argument a path from a source to a destination node, which is an array of triples composed of the geometry, the maximum speed, and the category of an edge of the path. The other arguments are the timestamp at which the trip starts and a Boolean value determining whether the points composed the trip are disturbed to simulate GPS errors. The output of the function is a temporal geometry point following this path. The procedure loops for each edge of the path and determines the number of segments of the edge, where a segment is a straight line defined by two consecutive points. For each segment, we determine the angle between the current segment and the next one (if any) to compute the maximum speed at the turn. This is determined by multiplying the maximum speed of the segment by a factor proportional to the angle so that the factor is 1.00 at both 0° and 360° and is 0.0 at 180°. Examples of values of degrees and the associated factor are given next.

```
0: 1.00, 5: 0.97, 45: 0.75, 90: 0.50, 135: 0.25, 175: 0.03
180: 0.00, 185: 0.03, 225: 0.25, 270: 0.50, 315: 0.75, 355: 0.97, 360: 0.00
```

Each segment is divided in fractions of length `P_EVENT_LENGTH`, which is by default 5 meters. We then loop for each fraction and choose to add one event that can be an acceleration, a deceleration, or a stop event. If the speed of the vehicle is zero, only an accelation event can happen. For this, we increase the current speed with the value of `P_EVENT_ACC`, which is by default 12 Km/h, and verify that the speed is not greater than the maximum speed of either the edge or the next turn for the last fraction. Otherwise, if the current speed is not zero, we apply a deceleration or a stop event with a probability proportional to the maximum speed of the edge, otherwise we apply an acceleration event. After applying the event, if the speed is zero we add a waiting time with a random exponential distribution with mean `P_DEST_EXPMU`, which is by default 1 second. Otherwise, we move the current position towards the end of the segment and, depending on the variable `disturbData`, we disturbe the new position to simulate GPS errors. The timestamp at which the vehicle reaches the new position is determined by dividing the distance traversed by the current speed. Finally, at the end of each segment, if the current speed is not zero, we add a stop event depending on the categories of the current segment and the next one. This is determined by a transition matrix given by the parameter `P_DEST_STOPPROB`.

2.8 Customizing the Generator to Your City

In order to customize the generator to a particular city the only thing we need is to define a bounding box that will be used to download the data from OSM. There are many ways to obtain such a bounding box, and a typical way to proceed is to use one of the multiple online services that allows one to visually define a bounding box over a map. Figure 2.5 shows how we can define the bounding box around Barcelona using the web site bboxfinder.

After obtaining the bounding box, we can proceed as we stated in Section 2.4. We create a new database `barcelona`, then add both PostGIS, MobilityDB, and pgRouting to it.

```
CREATE EXTENSION mobilitydb CASCADE;
CREATE EXTENSION pgRouting;
```

Then, we download the OSM data from Barcelona using the Overpass API by writing the following in a terminal:

```
CITY="barcelona"
BBOX="2.042084,41.267743,2.258720,41.445043"
wget -O "$CITY.osm" "http://www.overpass-api.de/api/xapi?*[bbox=${BBOX}][@meta]"
```

We can optionally reduce the size of the OSM file as follows

```
sed -r "s/version=\"[0-9]+\" timestamp=\"[^\"]+\" changeset=\"[0-9]+\" uid=\"[0-9]+\"
user=\"[^\"]+\"//g" barcelona.osm -i.org
```

Finally, we load the map and convert it into a routable format suitable for pgRouting as follows.

```
osm2pgrouting -f barcelona.osm --dbname barcelona -c mapconfig.xml
```

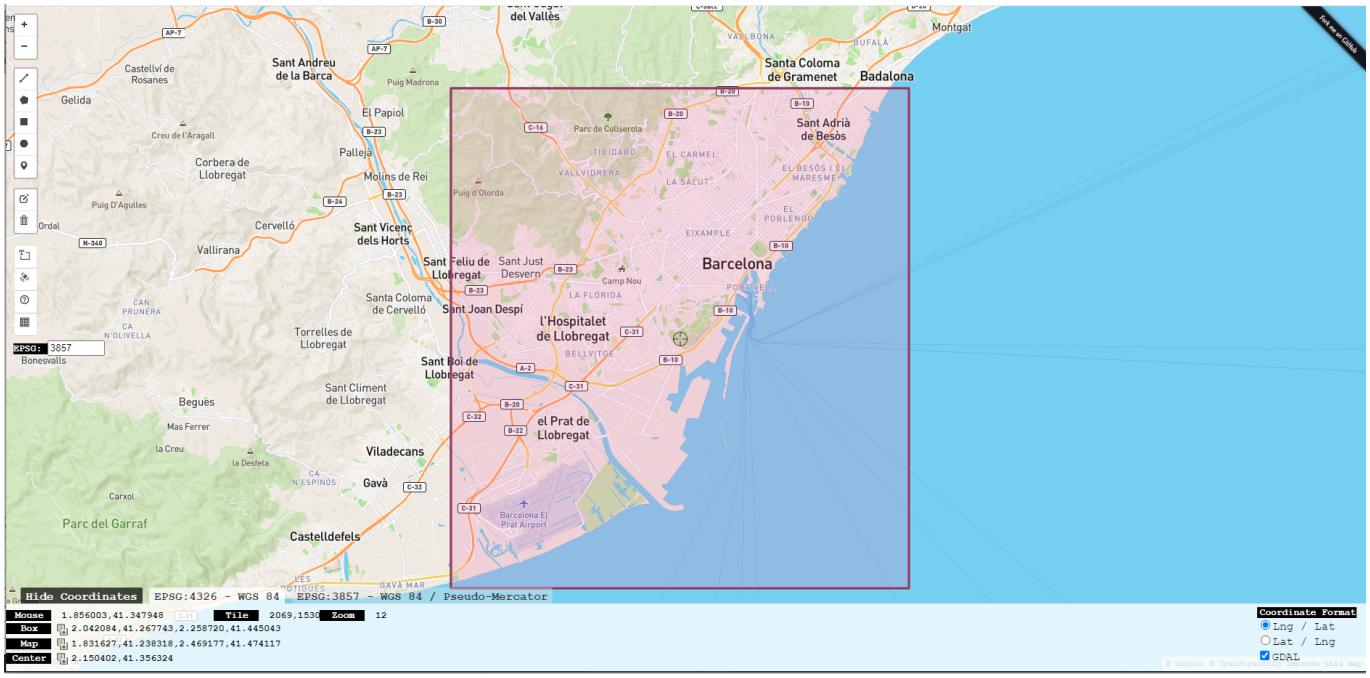

Figure 2.5: Defining the bounding box for obtaining OSM data from Barcelona.

2.9 Tuning the Generator Parameters

Multiple parameters can be used to tune the generator according to your needs. We describe next these parameters.

A first set of primary parameters determine the global behaviour of the generator. These parameters can also be set by a corresponding optional argument when calling the function `berlinmod_generate`.

- `P_SCALE_FACTOR: float`: Main parameter that determines the size of the data generated. Default value: 0.005. Corresponding optional argument: `scaleFactor`. By default, the scale factor determine the number of vehicles and the number of days they are observed as follows:

```
noVehicles int = round((2000 * sqrt(P_SCALE_FACTOR))::numeric, 0)::int;
noDays int = round((sqrt(P_SCALE_FACTOR) * 28)::numeric, 0)::int;
```

  For example, for a scale factor of 1.0, the number of vehicles and the number of days will be, respectively, 2000 and 28. Alternatively, you can manually set the number of vehicles or the number of days using the optional arguments `noVehicles` and `noDays`, which are both integers.

- `P_START_DAY: date`: The day the observation starts. Default value: Monday 2020-01-06. Corresponding optional argument: `startDay`.

- `P_PATH_MODE: text`: Method for selecting a path between source and target nodes. Possible values are `'Fastest Path'` (default) and `'Shortest Path'`. Corresponding optional argument: `pathMode`.

- `P_NODE_CHOICE: text`: Method for selecting home and work nodes. Possible values are `'Network Based'` for chosing the nodes with a uniform distribution among all nodes (default) and `'Region Based'` to use the population and number of enterprises statistics in the `Regions` tables. Corresponding optional argument: `nodeChoice`.

- `P_DISTURB_DATA: boolean`: Determine whether imprecision is added to the data generated. Possible values are false (no imprecision, default) and true (disturbed data). Corresponding optional argument: `disturbData`.

- `P_MESSAGES: text`: Quantity of messages shown describing the generation process. Possible values are `'minimal'`, `'mediummmm'`, `'verbose'`, and `'debug'`. Corresponding optional argument: `messages`.

- `P_TRIP_GENERATION: text`: Determine the language used to generate the trips. Possible values are `'C'` (default) and `'SQL'`. Corresponding optional argument: `tripGeneration`. This parameter determines whether the SQL function `createTrip` in the file `berlinmod_datagenerator.sql` or the function `createTripC`, which calls the trip generator written in C in MobilityDB (`createTrip` in MobilityDB 1.4, `create_trip` in MobilityDB 1.3), will be used for generating trips. Although the C function is faster than the corresponding SQL function, the SQL function can be easily modified to further customize the data generation.

For example, possible calls of the `berlinmod_generate` function setting values for the parameters are as follows.

```
-- Use all default values
SELECT berlinmod_generate();
-- Set the scale factor and use all other default values
SELECT berlinmod_generate(scaleFactor := 2.0);
-- Set the number of vehicles and number of days
SELECT berlinmod_generate(noVehicles := 10, noDays := 10);
```

Another set of parameters determining the global behaviour of the generator are given next.

- `P_RANDOM_SEED: float`: Seed for the random generator used to ensure deterministic results. Default value: 0.5.
- `P_NEIGHBOURHOOD_RADIUS: float`: Radius in meters defining a node neigbourhood. Default value: 3000.0.
- `P_SAMPLE_SIZE: int`: Size for sample relations. Default value: 100.
- `P_VEHICLE_TYPES: text[]`: Set of vehicle types. Default value: `{"passenger", "bus", "truck"}`.
- `P_VEHICLE_MODELS: text[]`: Set of vehicle models. Default value:

```
{"Mercedes-Benz", "Volkswagen", "Maybach", "Porsche", "Opel", "BMW", "Audi", "Acabion",
"Borgward", "Wartburg", "Sachsenring", "Multicar"}
```

- `P_PGROUTING_BATCH_SIZE: int`: Number of paths sent in a batch to pgRouting. Default value: 1e5 .

Another set of paramaters determine how the trips are created out of the paths.

- `P_EPSILON_SPEED: float`: Minimum speed in Km/h that is considered as a stop and thus only an accelaration event can be applied. Default value: 1.0.
- `P_EPSILON: float`: Minimum distance in the units of the coordinate system that is considered as zero. Default value: 0.0001.
- `P_EVENT_C: float`: The probability of a stop or a deceleration event is proportional to `P_EVENT_C / maxspeed`. Default value: 1.0
- `P_EVENT_P: float`: The probability for an event to be a stop. The complement 1.0 - `P_EVENT_P` is the probability for an event to be a deceleration. Default value: 0.1
- `P_EVENT_LENGTH: float`: Sampling distance in meters at which an acceleration, deceleration, or stop event may be generated. Default value: 5.0.
- `P_EVENT_ACC: float`: Constant speed in Km/h that is added to the current speed in an acceleration event. Default value: 12.0.
- `P_DEST_STOPPROB: float`: Probabilities for forced stops at crossings depending on the road type. It is defined by a transition matrix where lines and columns are ordered by side road (S), main road (M), freeway (F). The OSM highway types must be mapped to one of these categories in the function `berlinmod_roadCategory`. Default value:

```
{{0.33, 0.66, 1.00}, {0.33, 0.50, 0.66}, {0.10, 0.33, 0.05}}
```

- `P_DEST_EXPMU: float`: Mean waiting time in seconds using an exponential distribution. Increasing/decreasing this parameter allows us to slow down or speed up the trips. Could be think of as a measure of network congestion. Given a specific path, fine-tuning this parameter enable us to obtain an average travel time for this path that is the same as the expected travel time computed by a routing service such as, e.g., Google Maps. Default value: 1.0.
- `P_GPS_TOTALMAXERR: float` and `P_GPS_STEPMAXERR: float`: Parameters for simulating measuring errors. They are only required when the parameter `P_DISTURB_DATA` is true. They are, respectively, the maximum total deviation from the real position and maximum deviation per step, both in meters. Default values: 100.0 and 1.0.

2.10 Changing the Simulation Scenario

In this workshop, we have used until now the BerlinMOD scenario, which models the trajectories of persons going from home to work in the morning and returning back from work to home in the evening during the week days, with one possible leisure trip during the weekday nights and two possible leisure trips in the morning and in the afternoon of the weekend days. In this section, we devise another scenario for the data generator. This scenario corresponds to a home appliance shop that has several warehouses located in various places of the city. From each warehouse, the deliveries of appliances to customers are done by vehicles belonging to the warehouse. Although this scenario is different than BerlinMOD, many things can be reused and adapted. For example, home nodes can be replaced by warehouse locations, leisure destinations can be replaced by customer locations, and in this way many functions of the BerlinMOD SQL code will work directly. This is a direct benefit of having the simulation code written in SQL, so it will be easy to adapt to other scenarios. We describe next the needed changes.

Each day of the week excepted Sundays, deliveries of appliances from the warehouses to the customers are organized as follows. Each warehouse has several vehicles that make the deliveries. To each vehicle is assigned a list of customers that must be delivered during a day. A trip for a vehicle starts and ends at the warehouse and make the deliveries to the customers in the order of the list. Notice that in a real-world situation, the scheduling of the deliveries requires to take into account customers' availability in a time slot of a day and the time needed to make the delivery of the previous customers in the list. We do not take into account these aspects in this simple simulation scenario.

To be able to run the delivery generator you need to execute the first two steps specified in Section 2.4 to load the street network and prepare the base data for simulation, if not done already. The delivery generator can then be run as follows.

```
psql -h localhost -p 5432 -U dbowner -d brussels -f deliveries_datagenerator.sql
# adds the pgplsql functions of the simulation to the database

psql -h localhost -p 5432 -U dbowner -d brussels \
-c 'select deliveries_datagenerator(scaleFactor := 0.005)'
# calls the main pgplsql function to start the simulation
```

If everything is correct, you should see an output like that starts with this:

```
INFO:  ------------------------------------------------------------------
INFO:  Starting deliveries generation with scale factor 0.005
INFO:  ------------------------------------------------------------------
INFO:  Parameters:
INFO:  ------------
INFO:  No. of warehouses = 7, No. of vehicles = 141, No. of days = 4
INFO:  Start day = 2020-06-01, Path mode = Fastest Path, Disturb data = f
...
```

The generator will take about one minute. It will generate deliveries, according to the default parameters, for 141 cars over 2 days starting from Monday, June 1st 2020. It is possible to generate more or less data by respectively passing a bigger or a smaller scale factor value. Please refer to the Section 2.8 to see all the parameters that can be used to customize the simulation, with the exception of the `P_NEIGHBOURHOOD_RADIUS` parameter, which is not used in this scenario.

We describe next the main steps in the generation of the deliveries scenario.

We start by generating the `Warehouses` table. Each warehouse is located at a random node of the network.

```
DROP TABLE IF EXISTS Warehouses;
CREATE TABLE Warehouses(WarehouseId int, NodeId bigint, Geom geometry(Point));
```

```
FOR i IN 1..noWarehouses LOOP
  INSERT INTO Warehouses(WarehouseId, NodeId, Geom)
  SELECT i, id, Geom
  FROM Nodes n
  ORDER BY id LIMIT 1 OFFSET random_int(1, noNodes);
END LOOP;
```

We create a table `Vehicles` with all vehicles and the associated warehouse. Warehouses are associated to vehicles in a round-robin way.

```
DROP TABLE IF EXISTS Vehicles;
CREATE TABLE Vehicles(VehicleId int PRIMARY KEY, Licence text, VehicleType text,
  Brand text, WarehouseId int);
FOR i IN 1..noVehicles LOOP
  licence = berlinmod_createLicence(i);
  type = VEHICLETYPES[random_int(1, NOVEHICLETYPES)];
  brand = NOVEHICLEBRANDS[random_int(1, NOVEHICLEBRANDS)];
  warehouse = 1 + ((i - 1) % noWarehouses);
  INSERT INTO Vehicles VALUES (i, licence, type, brand, warehouse);
END LOOP;
```

We create next the `Trips` and `Destinations` tables that contain, respectively, the list of source and destination nodes composing the delivery trip of a vehicle for a day, and the list of source and destination nodes for all vehicles.

```
DROP TABLE IF EXISTS Trips;
CREATE TABLE Trips(VehicleId int, day date, seq int, source bigint, target bigint,
  PRIMARY KEY (VehicleId, day, seq));
DROP TABLE IF EXISTS Destinations;
CREATE TABLE Destinations(id serial PRIMARY KEY, source bigint, target bigint);
-- Loop for every vehicle
FOR i IN 1..noVehicles LOOP
  -- Get the warehouse node
  SELECT w.node INTO warehouseNode
  FROM Vehicles v, Warehouses w
  WHERE v.vehicleId = i AND v.warehouse = w.WarehouseId;
  day = startDay;
  -- Loop for every generation day
  FOR j IN 1..noDays LOOP
    -- Generate delivery trips excepted on Sunday
    IF date_part('dow', day) <> 0 THEN
      -- Select a number of destinations between 3 and 7
      SELECT random_int(3, 7) INTO noDest;
      sourceNode = warehouseNode;
      prevNodes = '{}';
      FOR k IN 1..noDest + 1 LOOP
        IF k <= noDest THEN
          targetNode = deliveries_selectDestNode(i, noNodes, prevNodes);
          prevNodes = prevNodes || targetNode;
        ELSE
          targetNode = warehouseNode;
        END IF;
        IF targetNode IS NULL THEN
          RAISE EXCEPTION '    Destination node cannot be NULL';
        END IF;
        IF sourceNode = targetNode THEN
          RAISE EXCEPTION '    SourceNode and destination nodes must be different, node: %' ↩
              , sourceNode;
        END IF;
        -- Keep the start and end nodes of each subtrip
        INSERT INTO Segments VALUES (i, day, k, sourceNode, targetNode);
        INSERT INTO Destinations(source, target) VALUES (sourceNode, targetNode);
        sourceNode = targetNode;
```

```
      END LOOP;
    END IF;
    day = day + interval '1 day';
  END LOOP;
END LOOP;
```

For every vehicle and every day which is not Sunday we proceed as follows. We randomly chose a number between 3 and 7 destinations and call the function `deliveries_selectDestNode` for determining these destinations. This function choses a destination node which is different from the previous nodes of the delivery, which are kept in the variable `prevNodes`. Then, the sequence of source and destination couples starting in the warehouse, visiting sequentially the clients to deliver and returning to the warehouse are added to the tables `Segments` and `Destinations`.

Next, we compute the paths between all source and target nodes that are in the `Destinations` table. Such paths are generated by pgRouting and stored in the `Paths` table.

```
DROP TABLE IF EXISTS Paths;
CREATE TABLE Paths(seq int, path_seq int, start_vid bigint, end_vid bigint,
  node bigint, edge bigint, cost float, agg_cost float,
  -- These attributes are filled in the subsequent update
  Geom geometry, speed float, category int);
-- Select query sent to pgRouting
IF pathMode = 'Fastest Path' THEN
  query1_pgr = 'SELECT id, source, target, cost_s AS cost, '
    'reverse_cost_s as reverse_cost FROM edges';
ELSE
  query1_pgr = 'SELECT id, source, target, length_m AS cost, '
    'length_m * sign(reverse_cost_s) as reverse_cost FROM edges';
END IF;
-- Get the total number of paths and number of calls to pgRouting
SELECT COUNT(*) INTO noPaths FROM (SELECT DISTINCT source, target FROM Destinations) AS t;
noCalls = ceiling(noPaths / P_PGROUTING_BATCH_SIZE::float);
FOR i IN 1..noCalls LOOP
  query2_pgr = format('SELECT DISTINCT source, target FROM Destinations '
    'ORDER BY source, target LIMIT %s OFFSET %s',
    P_PGROUTING_BATCH_SIZE, (i - 1) * P_PGROUTING_BATCH_SIZE);
  INSERT INTO Paths(seq, path_seq, start_vid, end_vid, node, edge, cost, agg_cost)
  SELECT * FROM pgr_dijkstra(query1_pgr, query2_pgr, true);
END LOOP;
UPDATE Paths SET
  -- adjusting directionality
  Geom = CASE WHEN node = e.source THEN e.Geom ELSE ST_Reverse(e.Geom) END,
  speed = maxspeed_forward,
  category = berlinmod_roadCategory(tag_id)
FROM Edges e WHERE e.id = edge;
```

After creating the `Paths` table, we set the query to be sent to pgRouting depending on whether we have want to compute the fastest or the shortest paths between two nodes. The generator uses the parameter `P_PGROUTING_BATCH_SIZE` to determine the maximum number of paths we compute in a single call to pgRouting. This parameter is set to 10,000 by default. Indeed, there is limit in the number of paths that pgRouting can compute in a single call and this depends in the available memory of the computer. Therefore, we need to determine the number of calls to pgRouting and compute the paths by calling the function `pgr_dijkstra`. Finally, we need to adjust the directionality of the geometry of the edges depending on which direction a trip traverses the edges, and set the speed and the category of the edges.

We explain how to generate the trips for a number of vehicles and a number of days starting at a given day.

```
DROP TABLE IF EXISTS Deliveries;
CREATE TABLE Deliveries(DeliveryId int PRIMARY KEY, VehicleId int, Day date, noCustomers ↩
    int,
  Trip tgeompoint, Trajectory geometry);
DROP TABLE IF EXISTS Segments;
CREATE TABLE Segments(DeliveryId int, seq int, source bigint, target bigint,
  Trip tgeompoint,
```

```
  -- These columns are used for visualization purposes
  trajectory geometry, sourceGeom geometry, PRIMARY KEY (DeliveryId, seq));
delivId = 1;
aDay = startDay;
FOR i IN 1..noDays LOOP
  SELECT date_part('dow', aDay) into weekday;
  -- 6: saturday, 0: sunday
  IF weekday <> 0 THEN
    FOR j IN 1..noVehicles LOOP
      -- Start delivery
      t = aDay + time '07:00:00' + createPauseN(120);
      -- Get the number of segments (number of destinations + 1)
      SELECT count(*) INTO noSegments
      FROM Trips
      WHERE VehicleId = j AND day = aDay;
      FOR k IN 1..noSegments LOOP
        -- Get the source and destination nodes of the segment
        SELECT source, target INTO sourceNode, targetNode
        FROM Trips
        WHERE VehicleId = j AND day = aDay AND seq = k;
        -- Get the path
        SELECT array_agg((Geom, speed, category) ORDER BY path_seq) INTO Path
        FROM Paths p
        WHERE start_vid = sourceNode AND end_vid = targetNode AND edge > 0;
        IF Path IS NULL THEN
          RAISE EXCEPTION 'The path of a trip cannot be NULL. '
            'SourceNode node: %, target node: %, k: %, noSegments: %', sourceNode,
              targetNode, k, noSegments;
        END IF;
        startTime = t;
        trip = createTripC(Path, t, disturbData, messages);
        IF trip IS NULL THEN
          RAISE EXCEPTION 'A trip cannot be NULL';
        END IF;
        t = endTimestamp(trip);
        tripTime = t - startTime;
        IF k < noSegments THEN
          -- Add a delivery time in [10, 60] min using a bounded Gaussian distribution
          deliveryTime = random_boundedgauss(10, 60) * interval '1 min';
          t = t + deliveryTime;
          trip = appendInstant(trip, tgeompoint_inst(endValue(trip), t));
        END IF;
        alltrips = alltrips || trip;
        SELECT Geom INTO sourceGeom FROM Nodes WHERE id = sourceNode;
        INSERT INTO Segments(DeliveryId, SeqNo, Source, target, trip, trajectory,  ↩
            sourceGeom)
          VALUES (delivId, k, sourceNode, targetNode, trip, trajectory(trip), sourceGeom);
      END LOOP;
      trip = merge(alltrips);
      INSERT INTO Deliveries(DeliveryId, VehicleId, day, noCustomers, trip, trajectory)
        VALUES (delivId, j, aDay, noSegments - 1, trip, trajectory(trip));
      delivId = delivId + 1;
      alltrips = '{}';
    END LOOP;
  END IF;
  aDay = aDay + interval '1 day';
END LOOP;
```

We start by creating the tables `Deliveries` and `Segments`. Then, the procedure simply loops for each day (excepted Sundays) and for each vehicle and generates the deliveries. For this, we first set the start time of a delivery trip by adding to 7 am a random non-zero duration of 120 minutes using a uniform distribution. Then, for every couple of source and destination

nodes in a segment, we call the function `createTrip` that we have seen previously to generate the Trip. We add a delivery time between 10 and 60 minutes using a bounded Gaussian distribution before starting the trip to the next customer or the return trip to the warehouse and then insert the trip into the `Segments` table.

Figure 2.6 and Figure 2.7 show visualizations of the data generated for the deliveries scenario.

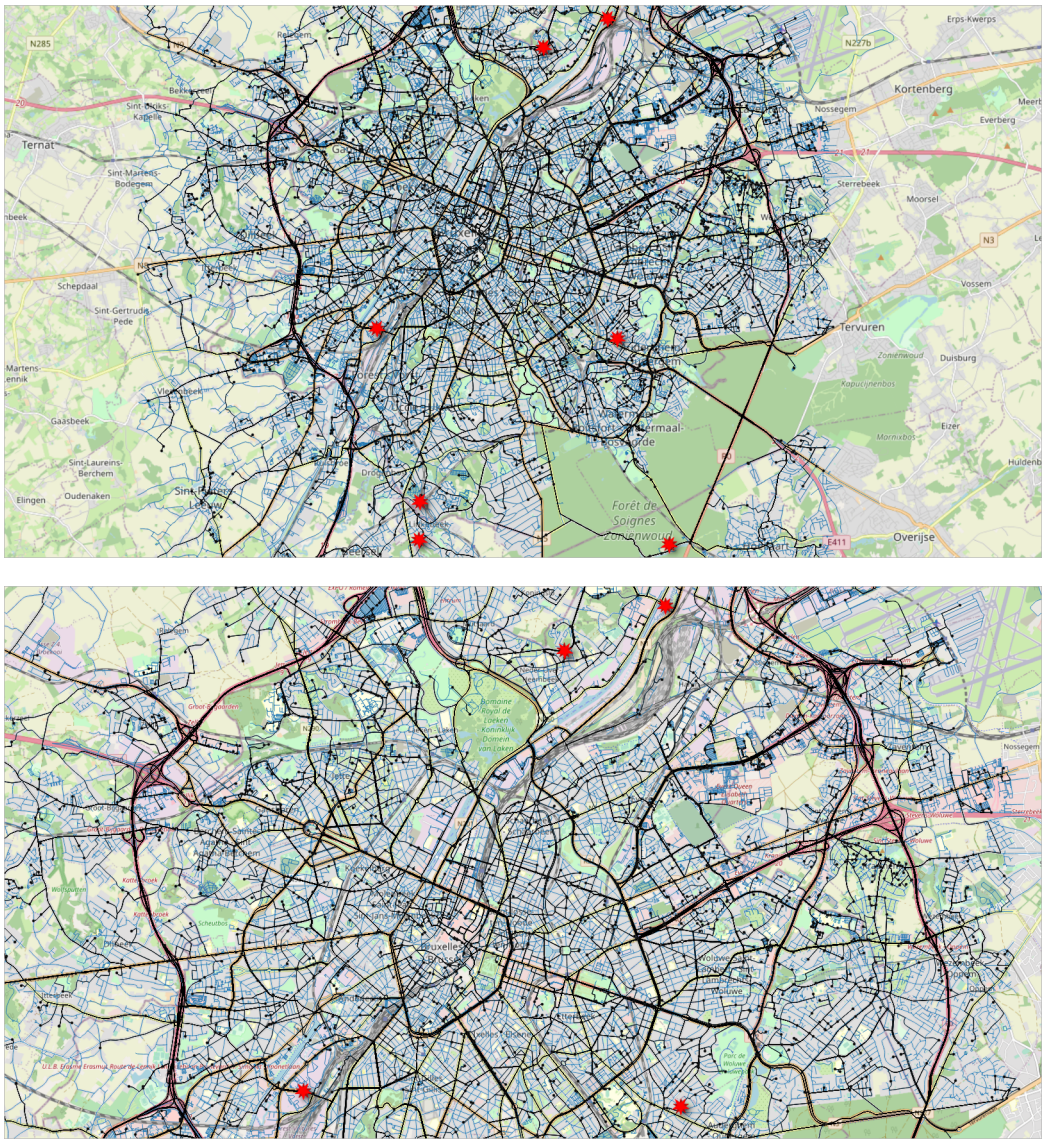

Figure 2.6: Visualization of the data generated for the deliveries scenario. The road network is shown with blue lines, the warehouses are shown with a red star, the routes taken by the deliveries are shown with black lines, and the location of the customers with black points.

2.11 Creating a Graph from Input Data

In this workshop, we have used until now the network topology obtained by osm2pgrouting. However, in some circumstances it is necessary to build the network topology ourselves, for example, when the data comes from other sources than OSM, such as data from an official mapping agency. In this section we show how to build the network topology from input data. We import Brussels data from OSM into a PostgreSQL database using osm2pgsql. Then, we construct the network topology using SQL so that the resulting graph can be used with pgRouting. We show two approaches for doing this, depending on whether we want

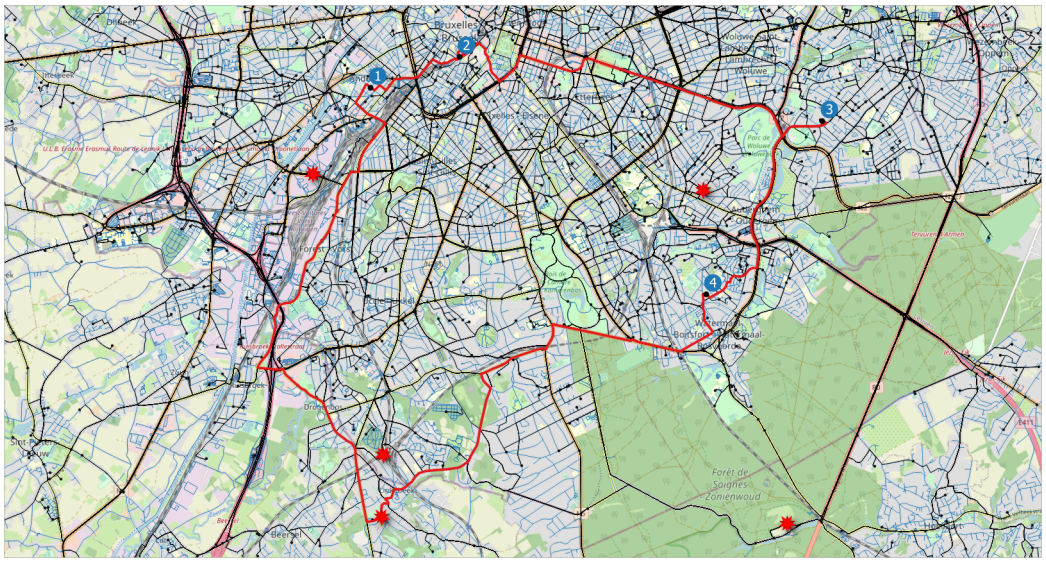

Figure 2.7: Visualization of the deliveries of one vehicle during one day. A delivery trip starts and ends at a warehouse and make the deliveries to several customers, four in this case.

to keep the original roads of the input data or we want to merge roads when they have similar characteristics such as road type, direction, maximum speed, etc. At the end, we compare the two networks obtained with the one obtained by osm2pgrouting.

2.11.1 Creating the Graph

As we did at the beginning of this chapter, we load the OSM data from Brussels into PostgreSQL with the following command.

```
osm2pgsql --create --database brussels --host localhost brussels.osm
```

The table `planet_osm_line` contains all linear features imported from OSM, in particular road data, but also many other features which are not relevant for our use case such as pedestrian paths, cycling ways, train ways, electric lines, etc. Therefore, we use the attribute `highway` to extract the roads from this table. We first create a table containing the road types we are interested in and associate to them a priority, a maximum speed, and a category as follows.

```
DROP TABLE IF EXISTS RoadTypes;
CREATE TABLE RoadTypes(id int PRIMARY KEY, type text, priority float, maxspeed float,
category int);
INSERT INTO RoadTypes VALUES
(101, 'motorway', 1.0, 120, 1),
(102, 'motorway_link', 1.0, 120, 1),
(103, 'motorway_junction', 1.0, 120, 1),
(104, 'trunk', 1.05, 120, 1),
(105, 'trunk_link', 1.05, 120, 1),
(106, 'primary', 1.15, 90, 2),
(107, 'primary_link', 1.15, 90, 1),
(108, 'secondary', 1.5, 70, 2),
(109, 'secondary_link', 1.5, 70, 2),
(110, 'tertiary', 1.75, 50, 2),
(111, 'tertiary_link', 1.75, 50, 2),
(112, 'residential', 2.5, 30, 3),
(113, 'living_street', 3.0, 20, 3),
(114, 'unclassified', 3.0, 20, 3),
(115, 'service', 4.0, 20, 3),
(116, 'services', 4.0, 20, 3);
```

Then, we create a table that contains the roads corresponding to one of the above types as follows.

```
DROP TABLE IF EXISTS Roads;
CREATE TABLE Roads AS
SELECT osm_id, admin_level, bridge, cutting, highway, junction, name, oneway, operator,
ref, route, surface, toll, tracktype, tunnel, width, way AS Geom
FROM planet_osm_line
WHERE highway IN (SELECT type FROM RoadTypes);

CREATE INDEX Roads_geom_idx ON Roads USING GiST(Geom);
```

We then create a table that contains all intersections between two roads as follows:

```
DROP TABLE IF EXISTS Intersections;
CREATE TABLE Intersections AS
WITH Temp1 AS (
  SELECT ST_Intersection(a.Geom, b.Geom) AS Geom
  FROM Roads a, Roads b
  WHERE a.osm_id < b.osm_id AND ST_Intersects(a.Geom, b.Geom) ),
Temp2 AS (
  SELECT DISTINCT Geom
  FROM Temp1
  WHERE geometrytype(Geom) = 'POINT'
  UNION
  SELECT (ST_DumpPoints(Geom)).Geom
  FROM Temp1
  WHERE geometrytype(Geom) = 'MULTIPOINT' )
SELECT ROW_NUMBER() OVER () AS id, Geom
FROM Temp2;

CREATE INDEX Intersections_geom_idx ON Intersections USING GIST(Geom);
```

The temporary table `Temp1` computes all intersections between two different roads, while the temporary table `Temp2` selects all intersections of type point and splits the intersections of type multipoint into the component points with the function `ST_DumpPoints`. Finally, the last query adds a sequence identifier to the resulting intersections.

Our next task is to use the table `Intersections` we have just created to split the roads. This is done as follows.

```
DROP TABLE IF EXISTS Segments;
CREATE TABLE Segments AS
SELECT DISTINCT osm_id, (ST_Dump(ST_Split(R.Geom, I.Geom))).Geom
FROM Roads R, Intersections I
WHERE ST_Intersects(R.Geom, I.Geom);

CREATE INDEX Segments_geom_idx ON Segments USING GIST(Geom);
```

The function `ST_Split` breaks the geometry of a road using an intersection and the function `ST_Dump` obtains the individual segments resulting from the splitting. However, as shown in the following query, there are duplicate segments with distinct `osm_id`.

```
SELECT S1.osm_id, S2.osm_id
FROM Segments S1, Segments S2
WHERE S1.osm_id < S2.osm_id AND ST_Intersects(S1.Geom, S2.Geom) AND
  ST_Equals(S1.Geom, S2.Geom);
-- 490493551  740404156
-- 490493551  740404157
```

We can remove those duplicates segments with the following query, which keeps arbitrarily the smaller `osm_id`.

```
DELETE FROM Segments S1
USING Segments S2
WHERE S1.osm_id > S2.osm_id AND ST_Equals(S1.Geom, S2.Geom);
```

We can obtain some characteristics of the segments with the following queries.

```
SELECT DISTINCT geometrytype(Geom) FROM Segments;
-- "LINESTRING"

SELECT MIN(ST_NPoints(Geom)), max(ST_NPoints(Geom)) FROM Segments;
-- 2  283
```

Now we are ready to obtain a first set of nodes for our graph.

```
DROP TABLE IF EXISTS TempNodes;
CREATE TABLE TempNodes AS
WITH Temp(Geom) AS (
  SELECT ST_StartPoint(Geom) FROM Segments UNION
  SELECT ST_EndPoint(Geom) FROM Segments )
SELECT ROW_NUMBER() OVER () AS id, Geom
FROM Temp;

CREATE INDEX TempNodes_geom_idx ON TempNodes USING GIST(Geom);
```

The above query select as nodes the start and the end points of the segments and assigns to each of them a sequence identifier. We construct next the set of edges of our graph as follows.

```
DROP TABLE IF EXISTS Edges;
CREATE TABLE Edges(id bigint, osm_id bigint, tag_id int, length_m float, source bigint,
  target bigint, cost_s float, reverse_cost_s float, one_way int, maxspeed float,
  priority float, Geom geometry);
INSERT INTO Edges(id, osm_id, source, target, Geom, length_m)
SELECT ROW_NUMBER() OVER () AS id, S.osm_id, n1.id AS source, n2.id AS target, S.Geom,
  ST_Length(S.Geom) AS length_m
FROM Segments S, TempNodes n1, TempNodes n2
WHERE ST_Intersects(ST_StartPoint(S.Geom), n1.Geom) AND
  ST_Intersects(ST_EndPoint(S.Geom), n2.Geom);

CREATE UNIQUE INDEX Edges_id_idx ON Edges USING BTREE(id);
CREATE INDEX Edges_geom_index ON Edges USING GiST(Geom);
```

The above query connects the segments obtained previously to the source and target nodes. We can verify that all edges were connected correctly to their source and target nodes using the following query.

```
SELECT count(*) FROM Edges WHERE source IS NULL OR target IS NULL;
-- 0
```

Now we can fill the other attributes of the edges. We start first with the attributes `tag_id`, `priority`, and `maxspeed`, which are obtained from the table `RoadTypes` using the attribute `highway`.

```
UPDATE Edges e
SET tag_id = t.id, priority = t.priority, maxspeed = t.maxSpeed
FROM Roads R, RoadTypes t
WHERE e.osm_id = R.osm_id AND R.highway = t.type;
```

We continue with the attribute `one_way` according to the semantics stated in the OSM documentation.

```
UPDATE Edges e
SET one_way = CASE
  WHEN R.oneway = 'yes' OR R.oneway = 'true' OR R.oneway = '1' THEN 1 -- Yes
  WHEN R.oneway = 'no' OR R.oneway = 'false' OR R.oneway = '0' THEN 2 -- No
  WHEN R.oneway = 'reversible' THEN 3 -- Reversible
  WHEN R.oneway = '-1' OR R.oneway = 'reversed' THEN -1 -- Reversed
  WHEN R.oneway IS NULL THEN 0 -- Unknown
  END
FROM Roads R
WHERE e.osm_id = R.osm_id;
```

We compute the implied one way restriction based on OSM documentation as follows.

```
UPDATE Edges e
SET one_way = 1
FROM Roads R
WHERE e.osm_id = R.osm_id AND R.oneway IS NULL AND
  (R.junction = 'roundabout' OR R.highway = 'motorway');
```

Finally, we compute the cost and reverse cost in seconds according to the length and the maximum speed of the edge.

```
UPDATE Edges e SET
cost_s = CASE
  WHEN one_way = -1 THEN - length_m / (maxspeed / 3.6)
  ELSE length_m / (maxspeed / 3.6)
  END,
reverse_cost_s = CASE
  WHEN one_way = 1 THEN - length_m / (maxspeed / 3.6)
  ELSE length_m / (maxspeed / 3.6)
  END;
```

Our last task is to compute the strongly connected components of the graph. This is necessary to ensure that there is a path between every couple of arbitrary nodes in the graph.

```
DROP TABLE IF EXISTS Nodes;
CREATE TABLE Nodes AS
WITH Components AS (
  SELECT * FROM pgr_strongComponents(
    'SELECT id, source, target, length_m AS cost, '
    'length_m * sign(reverse_cost_s) AS reverse_cost FROM Edges') ),
LargestComponent AS (
  SELECT component, count(*) FROM Components
  GROUP BY component ORDER BY count(*) DESC LIMIT 1 ),
Connected AS (
  SELECT Geom
  FROM TempNodes n, LargestComponent L, Components C
  WHERE n.id = C.node AND C.component = L.component )
SELECT ROW_NUMBER() OVER () AS id, Geom
FROM Connected;

CREATE UNIQUE INDEX Nodes_id_idx ON Nodes USING BTREE(id);
CREATE INDEX Nodes_geom_idx ON Nodes USING GiST(Geom);
```

The temporary table `Components` is obtained by calling the function `pgr_strongComponents` from pgRouting, the temporary table `LargestComponent` selects the largest component from the previous table, and the temporary table `Connected` selects all nodes that belong to the largest component. Finally, the last query assigns a sequence identifier to all nodes.

Now that we computed the nodes of the graph, we need to link the edges with the identifiers of these nodes. This is done as follows.

```
UPDATE Edges SET source = NULL, target = NULL;

UPDATE Edges e SET
  source = n1.id, target = n2.id
FROM Nodes n1, Nodes n2
WHERE ST_Intersects(e.Geom, n1.Geom) AND ST_StartPoint(e.Geom) = n1.Geom AND
  ST_Intersects(e.Geom, n2.Geom) AND ST_EndPoint(e.Geom) = n2.Geom;
```

We first set the identifiers of the source and target nodes to NULL before connecting them to the identifiers of the node. Finally, we delete the edges whose source or target node has been removed.

```
DELETE FROM Edges WHERE source IS NULL OR target IS NULL;
-- DELETE 1080
```

In order to compare the graph we have just obtained with the one obtained by osm2pgrouting we can issue the following queries.

```
SELECT count(*) FROM Ways;
-- 83017
SELECT count(*) FROM Edges;
-- 81073
SELECT count(*) FROM Ways_vertices_pgr;
-- 66832
SELECT count(*) FROM Nodes;
-- 45494
```

As can be seen, we have reduced the size of the graph. This can also be shown in Figure 2.8, where the nodes we have obtained are shown in blue and the ones obtained by osm2pgrouting are shown in red. It can be seen that osm2pgrouting adds many more nodes to the graph, in particular, at the intersection of a road and a pedestrian crossing. Our method only adds nodes when there is an intersection between two roads. We will show in the next section how this network can still be optimized by removing unnecessary nodes and merging the corresponding edges.

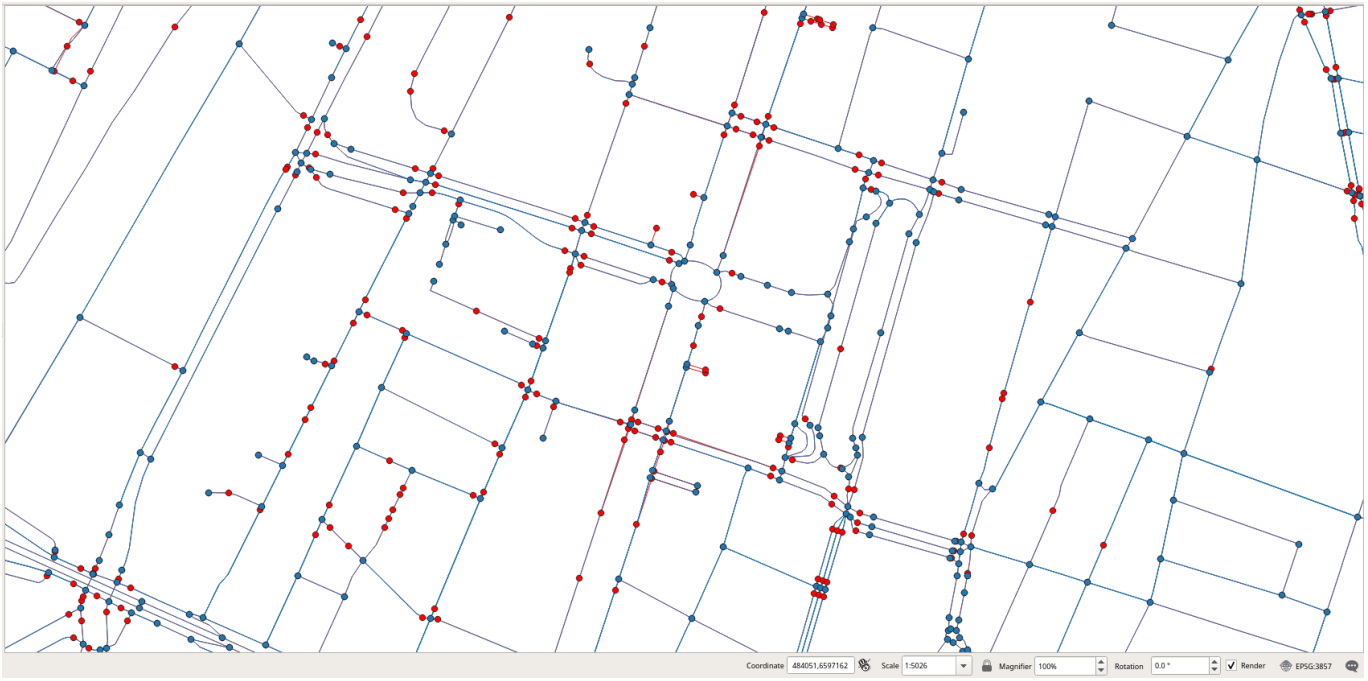

Figure 2.8: Comparison of the nodes obtained (in blue) with those obtained by osm2pgrouting (in red).

2.11.2 Linear Contraction of the Graph

We show next a possible approach to contract the graph. This approach corresponds to linear contraction provided by pgRouting although we do it differently by taking into account the type, the direction, and the geometry of the roads. For this, we get the initial roads to merge as we did previously but now we put them in a table `TempRoads`.

```
DROP TABLE IF EXISTS TempRoads;
CREATE TABLE TempRoads AS
SELECT osm_id, admin_level, bridge, cutting, highway, junction, name, oneway, operator,
  ref, route, surface, toll, tracktype, tunnel, width, way AS Geom
FROM planet_osm_line
WHERE highway IN (SELECT type FROM RoadTypes);
-- SELECT 37045
CREATE INDEX TempRoads_geom_idx ON TempRoads USING GiST(Geom);
```

Then, we use the following procedure to merge the roads.

```
CREATE OR REPLACE FUNCTION mergeRoads()
RETURNS void LANGUAGE PLPGSQL AS $$
DECLARE
  i integer = 1;
  cnt integer;
BEGIN
  -- Create tables
  DROP TABLE IF EXISTS MergedRoads;
  CREATE TABLE MergedRoads AS
  SELECT *, '{}'::bigint[] AS Path
  FROM TempRoads;
  CREATE INDEX MergedRoads_geom_idx ON MergedRoads USING GIST(Geom);
  DROP TABLE IF EXISTS Merge;
  CREATE TABLE Merge(osm_id1 bigint, osm_id2 bigint, Geom geometry);
  DROP TABLE IF EXISTS DeletedRoads;
  CREATE TABLE DeletedRoads(osm_id bigint);
  -- Iterate until no geometry can be extended
  LOOP
    RAISE INFO 'Iteration %', i;
    i = i + 1;
    -- Compute the union of two roads
    DELETE FROM Merge;
    INSERT INTO Merge
    SELECT R1.osm_id AS osm_id1, R2.osm_id AS osm_id2,
      ST_LineMerge(ST_Union(R1.Geom, R2.Geom)) AS Geom
    FROM MergedRoads R1, TempRoads R2
    WHERE R1.osm_id <> R2.osm_id AND R1.highway = R2.highway AND
      R1.oneway = R2.oneway AND ST_Intersects(R1.Geom, R2.Geom) AND
      ST_EndPoint(R1.Geom) = ST_StartPoint(R2.Geom) AND NOT EXISTS (
        SELECT * FROM TempRoads R3
        WHERE osm_id NOT IN (SELECT osm_id FROM DeletedRoads) AND
          R3.osm_id <> R1.osm_id AND R3.osm_id <> R2.osm_id AND
          ST_Intersects(R3.Geom, ST_StartPoint(R2.Geom)) ) AND
      geometryType(ST_LineMerge(ST_Union(R1.Geom, R2.Geom))) = 'LINESTRING'
      AND NOT St_Equals(ST_LineMerge(ST_Union(R1.Geom, R2.Geom)), R1.Geom);
    -- Exit if there is no more roads to extend
    SELECT count(*) INTO cnt FROM Merge;
    RAISE INFO 'Extended % roads', cnt;
    EXIT WHEN cnt = 0;
    -- Extend the geometries
    UPDATE MergedRoads R SET
      Geom = M.Geom,
      Path = R.Path || osm_id2
    FROM Merge M
    WHERE R.osm_id = M.osm_id1;
    -- Keep track of redundant roads
    INSERT INTO DeletedRoads
    SELECT osm_id2 FROM Merge
    WHERE osm_id2 NOT IN (SELECT osm_id FROM DeletedRoads);
  END LOOP;
  -- Delete redundant roads
  DELETE FROM MergedRoads R USING DeletedRoads M
  WHERE R.osm_id = M.osm_id;
  -- Drop tables
  DROP TABLE Merge;
  DROP TABLE DeletedRoads;
  RETURN;
END; $$
```

The procedure starts by creating a table `MergedRoads` obtained by adding a column `Path` to the table `TempRoads` created before. This column keeps track of the identifiers of the roads that are merged with the current one and is initialized to an empty

array. It also creates two tables `Merge` and `DeletedRoads` that will contain, respectively, the result of merging two roads, and the identifiers of the roads that will be deleted at the end of the process. The procedure then iterates while there is at least one road that can be extended with the geometry of another one to which it connects to. More precisely, a road can be extended with the geometry of another one if they are of the same type and the same direction (as indicated by the attributes `highway` and `one_way`), the end point of the road is the start point of the other road, and this common point is not a crossing, that is, there is no other road that starts and this common point. Notice that we only merge roads if their resulting geometry is a linestring and we avoid infinite loops by verifying that the merge of the two roads is different from the original geometry. After that, we update the roads with the new geometries and add the identifier of the road used to extend the geometry into the `Path` attribute and the `DeletedRoads` table. After exiting the loop, the procedure finishes by removing unnecessary roads.

The above procedure iterates 20 times for the largest segment that can be assembled, which is located in the ring-road around Brussels between two exits. It takes 15 minutes to execute in my laptop.

```
INFO:  Iteration 1
INFO:  Extended 3431 roads
INFO:  Iteration 2
INFO:  Extended 1851 roads
INFO:  Iteration 3
INFO:  Extended 882 roads
INFO:  Iteration 4
INFO:  Extended 505 roads
[...]
INFO:  Iteration 17
INFO:  Extended 3 roads
INFO:  Iteration 18
INFO:  Extended 2 roads
INFO:  Iteration 19
INFO:  Extended 1 roads
INFO:  Iteration 20
INFO:  Extended 0 roads
```

After we apply the above procedure to merge the roads, we are ready to create a new set of roads from which we can construct the graph.

```
CREATE TABLE Roads AS
SELECT osm_id || Path AS osm_id, admin_level, bridge, cutting, highway, junction, name,
  oneway, operator, ref, route, surface, toll, tracktype, tunnel, width, Geom
FROM MergedRoads;

CREATE INDEX Roads_geom_idx ON Roads USING GiST(Geom);
```

Notice that now the attribute `osm_id` is an array of OSM identifiers (which are big integers), whereas in the previous section it was a single big integer.

We then proceed as we did in Section 2.11.1 to compute the set of nodes and the set of edges, which we will store now for comparison purposes into tables `Nodes1` and `Edges1`. We can issue the following queries to compare the two graphs we have obtained and the one obtained by osm2pgrouting .

```
SELECT count(*) FROM Ways;
-- 83017
SELECT count(*) FROM Edges;
-- 81073
SELECT count(*) FROM Edges1;
-- 77986
SELECT count(*) FROM Ways_vertices_pgr;
-- 66832
SELECT count(*) FROM Nodes;
-- 45494
SELECT count(*) FROM Nodes1;
-- 42156
```

Figure 2.9 shows the nodes for the three graphs, those obtained after contracting the graph are shown in black, those before contraction are shown in blue, and those obtained by osm2pgrouting are shown in red. The figure shows in particular how several segments of the ring-road around Brussels are merged together since the have the same road type, direction, and maximum speed, The figure also shows in read a road that was removed since it does not belong to the strongly connected components of the graph.

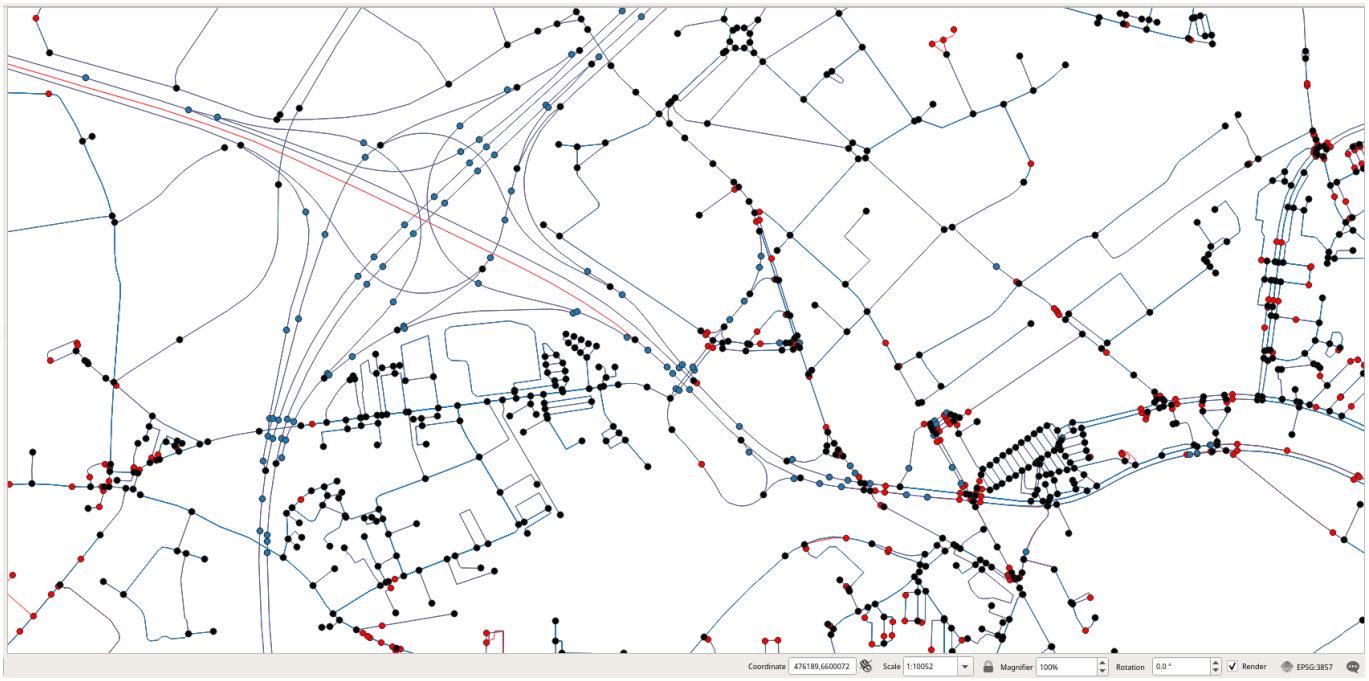

Figure 2.9: Comparison of the nodes obtained by contracting the graph (in black), before contraction (in blue), and those obtained by osm2pgrouting (in red).

Chapter 3

BerlinMOD Benchmark on MobilityDB

BerlinMOD is a standard benchmark for moving object DBMSs. It provides a data generator, pregenerated benchmark data for different scale factors, and set of queries of two types: 17 range-style queries (called BerlinMOD/r), and 9 nearest-neighbours queries (called BerlinMOD/NN). The MobilityDB tutorial presented in Chapter 1 and its associated data were based on BerlinMOD. However, its purpose was to show the capabilities of MobilityDB. In this chapter, we show how to load pregenerated BerlinMOD data on MobilityDB and how to express the 17 queries in BerlinMOD/r. Some of these queries were already presented in Chapter 1.

3.1 Loading the Data

The script for loading pregenerated data is available here.

```
-------------------------------------------------------------------------------
-- Loads the BerlinMOD data in projected (2D) coordinates with SRID 5676
-- https://epsg.io/5676
-------------------------------------------------------------------------------

DROP FUNCTION IF EXISTS berlinmod_load();
CREATE OR REPLACE FUNCTION berlinmod_load(scale_factor text DEFAULT '0.005',
  path text DEFAULT '/usr/local/BerlinMOD/')
RETURNS text AS $$
DECLARE
  fullpath text;
BEGIN
  fullpath = path || scale_factor || '/';
  DROP TABLE IF EXISTS streets;
  CREATE TABLE streets (
    StreetId integer,
    vmax integer,
    x1 double precision,
    y1 double precision,
    x2 double precision,
    y2 double precision,
    Geom geometry(LineString, 5676) );
  EXECUTE format('COPY streets(StreetId, vmax, x1, y1, x2, y2) FROM ''%sstreets.csv''
    DELIMITER '','' CSV HEADER', fullpath);
  UPDATE streets
  SET Geom = ST_Transform(ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(x1, y1),
    ST_MakePoint(x2, y2)]), 4326), 5676);

  DROP TABLE IF EXISTS Points CASCADE;
  CREATE TABLE Points (
    PointId integer,
```

```
  PosX double precision,
  PosY double precision,
  Geom geometry(Point, 5676) );
EXECUTE format('COPY Points(PointId, PosX, PosY) FROM ''%spoints.csv''
  DELIMITER  '','' CSV HEADER', fullpath);
UPDATE Points
SET Geom = ST_Transform(ST_SetSRID(ST_MakePoint(PosX, PosY), 4326), 5676);

CREATE INDEX Points_geom_idx ON Points USING gist(Geom);

CREATE VIEW Points1(PointId, PosX, PosY, Geom) AS
SELECT PointId, PosX, PosY, Geom
FROM Points
LIMIT 10;

DROP TABLE IF EXISTS RegionsInput CASCADE;
CREATE TABLE RegionsInput (
  RegionId integer,
  SegNo integer,
  XStart double precision,
  YStart double precision,
  XEnd double precision,
  YEnd double precision );
EXECUTE format('COPY RegionsInput(RegionId, SegNo, XStart, YStart, XEnd, YEnd)
  FROM ''%sregions.csv'' DELIMITER  '','' CSV HEADER', fullpath);

DROP TABLE IF EXISTS Regions CASCADE;
CREATE TABLE Regions (
  RegionId integer,
  Geom Geometry(Polygon, 5676) );
INSERT INTO Regions (RegionId, Geom)
WITH RegionsSegs AS (
  SELECT RegionId, SegNo, ST_Transform(ST_SetSRID(St_MakeLine(
    ST_MakePoint(XStart, YStart), ST_MakePoint(XEnd, YEnd)), 4326), 5676) AS Geom
  FROM RegionsInput )
SELECT RegionId, ST_Polygon(ST_LineMerge(ST_Union(Geom ORDER BY SegNo)), 5676) AS Geom
FROM RegionsSegs
GROUP BY RegionId;

CREATE INDEX Regions_geom_idx ON Regions USING gist(Geom);

CREATE VIEW Regions1(RegionId, Geom) AS
SELECT RegionId, Geom
FROM Regions
LIMIT 10;

DROP TABLE IF EXISTS Instants CASCADE;
CREATE TABLE Instants (
  InstantId integer,
  Instant timestamptz );
EXECUTE format('COPY Instants(InstantId, Instant) FROM ''%sinstants.csv''
  DELIMITER  '','' CSV HEADER', fullpath);

CREATE INDEX Instants_instant_btree_idx ON Instants USING btree(instant);

CREATE VIEW Instants1(InstantId, Instant) AS
SELECT InstantId, Instant
FROM Instants
LIMIT 10;

DROP TABLE IF EXISTS Periods CASCADE;
CREATE TABLE Periods (
```

```
  PeriodId integer,
  BeginP timestamp,
  EndP timestamp,
  Period tstzspan );
EXECUTE format('COPY Periods(PeriodId, BeginP, EndP) FROM ''%speriods.csv''
  DELIMITER  '','' CSV HEADER', fullpath);
UPDATE Periods
SET Period = tstzspan(BeginP,EndP);

CREATE INDEX Periods_Period_gist_idx ON Periods USING gist(Period);

CREATE VIEW Periods1(PeriodId, BeginP, EndP, Period) AS
SELECT PeriodId, BeginP, EndP, Period
FROM Periods
LIMIT 10;

DROP TABLE IF EXISTS Vehicles CASCADE;
CREATE TABLE Vehicles (
  VehicleId integer PRIMARY KEY,
  Licence varchar(32),
  VehicleType varchar(32),
  Model varchar(32) );
EXECUTE format('COPY Vehicles(VehicleId, Licence, VehicleType, Model) FROM ''%svehicles. ↩
    csv''
  DELIMITER  '','' CSV HEADER', fullpath);

DROP TABLE IF EXISTS Licences CASCADE;
CREATE TABLE Licences (
  VehicleId integer PRIMARY KEY,
  LicenceId integer,
  Licence varchar(8) );
EXECUTE format('COPY Licences(Licence, LicenceId) FROM ''%slicences.csv''
  DELIMITER  '','' CSV HEADER', fullpath);
UPDATE Licences q
SET VehicleId = ( SELECT v.VehicleId FROM Vehicles v WHERE v.Licence = q.Licence );

CREATE INDEX Licences_VehId_idx ON Licences USING btree(VehicleId);

CREATE VIEW Licences1(LicenceId, Licence, VehicleId) AS
SELECT LicenceId, Licence, VehicleId
FROM Licences
LIMIT 10;

CREATE VIEW Licences2(LicenceId, Licence, VehicleId) AS
SELECT LicenceId, Licence, VehicleId
FROM Licences
LIMIT 10 OFFSET 10;

DROP TABLE IF EXISTS TripsInput CASCADE;
CREATE TABLE TripsInput (
  VehicleId integer,
  TripId integer,
  TStart timestamp without time zone,
  TEnd timestamp without time zone,
  XStart double precision,
  YStart double precision,
  XEnd double precision,
  YEnd double precision,
  Geom geometry(LineString) );
EXECUTE format('COPY TripsInput(VehicleId, TripId, TStart, TEnd, XStart, YStart, XEnd,  ↩
    YEnd)
  FROM ''%strips.csv'' DELIMITER  '','' CSV HEADER', fullpath);
```

```
  UPDATE TripsInput
  SET Geom = ST_Transform(ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(XStart, YStart),
    ST_MakePoint(XEnd, YEnd)]), 4326), 5676);

  DROP TABLE IF EXISTS TripsInputInstants;
  CREATE TABLE TripsInputInstants AS (
  SELECT VehicleId, TripId, TStart, XStart, YStart,
    ST_Transform(ST_SetSRID(ST_MakePoint(XStart, YStart), 4326), 5676) as Geom
  FROM TripsInput
  UNION ALL
  SELECT t1.VehicleId, t1.TripId, t1.TEnd, t1.XEnd, t1.YEnd,
    ST_Transform(ST_SetSRID(ST_MakePoint(t1.XEnd, t1.YEnd), 4326), 5676) as Geom
  FROM TripsInput t1 INNER JOIN (
    SELECT VehicleId, TripId, max(TEnd) as MaxTend
    FROM TripsInput
    GROUP BY VehicleId, TripId
  ) t2 ON t1.VehicleId = t2.VehicleId AND t1.TripId = t2.TripId AND t1.TEnd = t2.MaxTend );
  ALTER TABLE TripsInputInstants ADD COLUMN inst tgeompoint;
  UPDATE TripsInputInstants
  SET inst = tgeompoint_inst(Geom, TStart);

  DROP TABLE IF EXISTS Trips CASCADE;
  CREATE TABLE Trips (
    TripId integer PRIMARY KEY,
    VehicleId integer NOT NULL,
    Trip tgeompoint,
    Traj geometry,
    PRIMARY KEY (VehicleId, TripId),
    FOREIGN KEY (VehicleId) REFERENCES Vehicles(VehicleId));
  INSERT INTO Trips
  SELECT VehicleId, TripId, tgeompoint_seq(array_agg(inst ORDER BY TStart))
  FROM TripsInputInstants
  GROUP BY VehicleId, TripId;
  UPDATE Trips
  SET Traj = trajectory(Trip);

  CREATE INDEX Trips_VehId_idx ON Trips USING btree(VehicleId);
  CREATE INDEX Trips_gist_idx ON Trips USING gist(trip);

  DROP VIEW IF EXISTS Trips1;
  CREATE VIEW Trips1 AS
  SELECT * FROM Trips LIMIT 100;

  -- Drop temporary tables
  DROP TABLE RegionsInput;
  DROP TABLE TripsInput;
  DROP TABLE TripsInputInstants;

  RETURN 'The End';
END;
$$ LANGUAGE 'plpgsql';
```

The script above creates a procedure to load pregenerated BerlinMOD data (in CSV format and WGS84 coordinates) at various scale factors. The procedure has two parameters: the scale factor and the directory where the CSV files are located. It supposes by default that the scale factor is `0.005` and that the CSV files are located in the directory `/usr/local/BerlinMOD/<scale factor>/`. Notice that the procedure creates GiST indexes for the tables. Alternatively, SP-GiST indexes could be used. The procedure can be called, for example, as follows.

```
SELECT berlinmod_load('0.05');
```

3.2 Loading the Data in Partitioned Tables

As we discussed in Chapter 1, partioning allows one to split a large table into smaller physical pieces. We show next how to modify the scripts given in the previous section to take advantage of partioning. We will partition the `Trips` table by date using list partitioning, where each partitition will contain all the trips that start at a particular date. We will use the procedure `create_partitions_by_date` shown in Chapter 1 for automatically creating the partitions according to the date range of the corresponding scale factor.

```
[...]
DROP TABLE IF EXISTS TripsInput CASCADE;
CREATE TABLE TripsInput (
  VehicleId integer,
  TripId integer,
  TripDate date,
  TStart timestamp without time zone,
  TEnd timestamp without time zone,
  XStart double precision,
  YStart double precision,
  XEnd double precision,
  YEnd double precision,
  Geom geometry(LineString) );
EXECUTE format('COPY TripsInput(VehicleId, TripId, TStart, TEnd, XStart, YStart, XEnd, YEnd ↩
    )
FROM ''%strips.csv'' DELIMITER  '','' CSV HEADER', fullpath);
UPDATE TripsInput
SET Geom = ST_Transform(ST_SetSRID(ST_MakeLine(ARRAY[ST_MakePoint(XStart, YStart),
  ST_MakePoint(XEnd, YEnd)]), 4326), 5676);
UPDATE TripsInput t1
SET TripDate = t2.TripDate
FROM (SELECT DISTINCT TripId, date_trunc('day', MIN(TStart) OVER
  (PARTITION BY TripId)) AS TripDate FROM TripsInput) t2
WHERE t1.TripId = t2.TripId;
[...]
DROP TABLE IF EXISTS Trips CASCADE;
CREATE TABLE Trips (
  VehicleId integer NOT NULL,
  TripId integer NOT NULL,
  TripDate date,
  Trip tgeompoint,
  Traj geometry,
  PRIMARY KEY (VehicleId, TripId, TripDate),
  FOREIGN KEY (VehicleId) REFERENCES Vehicles (VehicleId)
) PARTITION BY LIST(TripDate);

-- Create the partitions
SELECT MIN(TripDate), MAX(TripDate) INTO mindate, maxdate FROM TripsInputInstants;
PERFORM create_partitions_by_date('Trips', mindate, maxdate);

INSERT INTO Trips(VehicleId, TripId, TripDate, Trip)
SELECT VehicleId, TripId, TripDate, tgeompoint_seq(array_agg(inst ORDER BY TStart))
FROM TripsInputInstants
GROUP BY VehicleId, TripId, TripDate;
UPDATE Trips
SET Traj = trajectory(Trip);

CREATE INDEX Trips_VehId_idx ON Trips USING btree(VehicleId);
CREATE UNIQUE INDEX Trips_pkey_idx ON Trips USING btree(VehicleId, TripId, TripDate);
CREATE INDEX Trips_gist_idx ON Trips USING gist(trip);
[...]
```

With respect to the script given in the previous section, we need to add an additional column `TripDate` to the tables `TripsInput`,

`TripsInputInstants` (not shown), and `Trips` that will be used for partitioning.

3.3 BerlinMOD/r Queries

The script for querying BerlinMOD data loaded in MobilityDB with the BerlinMOD/r queries is available here.

1. What are the models of the vehicles with licence plate numbers from `Licences`?

```
SELECT DISTINCT l.Licence, v.Model AS Model
FROM Vehicles v, Licences l
WHERE v.Licence = l.Licence;
```

2. How many vehicles exist that are passenger cars?

```
SELECT COUNT (DISTINCT Licence)
FROM Vehicles v
WHERE VehicleType = 'passenger';
```

3. Where have the vehicles with licences from `Licences1` been at each of the instants from `Instants1`?

```
SELECT DISTINCT l.Licence, i.InstantId, i.Instant AS Instant,
  valueAtTimestamp(t.Trip, i.Instant) AS Pos
FROM Trips t, Licences1 l, Instants1 i
WHERE t.VehicleId = l.VehicleId AND valueAtTimestamp(t.Trip, i.Instant) IS NOT NULL
ORDER BY l.Licence, i.InstantId;
```

4. Which vehicles have passed the points from `Points`?

```
SELECT DISTINCT p.PointId, p.Geom, v.Licence
FROM Trips t, Vehicles v, Points p
WHERE t.VehicleId = v.VehicleId AND t.Trip && stbox(p.Geom) AND
  ST_Intersects(trajectory(t.Trip), p.Geom)
ORDER BY p.PointId, v.Licence;
```

5. What is the minimum distance between places, where a vehicle with a licence from `Licences1` and a vehicle with a licence from `Licences2` have been?

```
SELECT l1.Licence AS Licence1, l2.Licence AS Licence2,
  MIN(ST_Distance(trajectory(t1.Trip), trajectory(t2.Trip))) AS MinDist
FROM Trips t1, Licences1 l1, Trips t2, Licences2 l2
WHERE t1.VehicleId = l1.VehicleId AND t2.VehicleId = l2.VehicleId AND
  t1.VehicleId < t2.VehicleId
GROUP BY l1.Licence, l2.Licence
ORDER BY l1.Licence, l2.Licence;
```

6. What are the pairs of trucks that have ever been as close as 10m or less to each other?

```
SELECT DISTINCT v1.Licence AS Licence1, v2.Licence AS Licence2
FROM Trips t1, Vehicles v1, Trips t2, Vehicles v2
WHERE t1.VehicleId = v1.VehicleId AND t2.VehicleId = v2.VehicleId AND
  t1.VehicleId < t2.VehicleId AND v1.VehicleType = 'truck' AND
  v2.VehicleType = 'truck' AND t1.Trip && expandSpace(t2.Trip, 10) AND
  eDwithin(t1.Trip, t2.Trip, 10.0)
ORDER BY v1.Licence, v2.Licence;
```

7. What are the licence plate numbers of the passenger cars that have reached the points from `Points` first of all passenger cars during the complete observation period?

```
WITH Timestamps AS (
  SELECT DISTINCT v.Licence, p.PointId, p.Geom,
    MIN(startTimestamp(atValues(t.Trip,p.Geom))) AS Instant
  FROM Trips t, Vehicles v, Points1 p
  WHERE t.VehicleId = v.VehicleId AND v.VehicleType = 'passenger' AND
    t.Trip && stbox(p.Geom) AND ST_Intersects(trajectory(t.Trip), p.Geom)
  GROUP BY v.Licence, p.PointId, p.Geom )
SELECT t1.Licence, t1.PointId, t1.Geom, t1.Instant
FROM Timestamps t1
WHERE t1.Instant <= ALL (
  SELECT t2.Instant
  FROM Timestamps t2
  WHERE t1.PointId = t2.PointId )
ORDER BY t1.PointId, t1.Licence;
```

8. What are the overall travelled distances of the vehicles with licence plate numbers from `Licences1` during the periods from `Periods1`?

```
SELECT l.Licence, p.PeriodId, p.Period, SUM(length(atTime(t.Trip, p.Period))) AS Dist
FROM Trips t, Licences1 l, Periods1 p
WHERE t.VehicleId = l.VehicleId AND t.Trip && p.Period
GROUP BY l.Licence, p.PeriodId, p.Period
ORDER BY l.Licence, p.PeriodId;
```

9. What is the longest distance that was travelled by a vehicle during each of the periods from `Periods`?

```
WITH Distances AS (
  SELECT p.PeriodId, p.Period, t.VehicleId,
    SUM(length(atTime(t.Trip, p.Period))) AS Dist
  FROM Trips t, Periods p
  WHERE t.Trip && p.Period
  GROUP BY p.PeriodId, p.Period, t.VehicleId )
SELECT PeriodId, Period, MAX(Dist) AS MaxDist
FROM Distances
GROUP BY PeriodId, Period
ORDER BY PeriodId;
```

10. When and where did the vehicles with licence plate numbers from `Licences1` meet other vehicles (distance < 3m) and what are the latter licences?

```
WITH Values AS (
  SELECT DISTINCT l1.Licence AS QueryLicence, l2.Licence AS OtherLicence,
    atTime(t1.Trip, getTime(atValues(tdwithin(t1.Trip, t2.Trip, 3.0), TRUE))) AS Pos
  FROM Trips t1, Licences1 l1, Trips t2, Licences2 l2
  WHERE t1.VehicleId = l1.VehicleId AND t2.VehicleId = l2.VehicleId AND
    t1.VehicleId < t2.VehicleId AND
    expandSpace(t1.Trip, 3) && expandSpace(t2.Trip, 3) AND
    eDwithin(t1.Trip, t2.Trip, 3.0) )
SELECT QueryLicence, OtherLicence, array_agg(Pos ORDER BY startTimestamp(Pos)) AS Pos
FROM Values
GROUP BY QueryLicence, OtherLicence
ORDER BY QueryLicence, OtherLicence;
```

11. Which vehicles passed a point from `Points1` at one of the instants from `Instants1`?

```
SELECT p.PointId, p.Geom, i.InstantId, i.Instant, v.Licence
FROM Trips t, Vehicles v, Points1 p, Instants1 i
WHERE t.VehicleId = v.VehicleId AND t.Trip @> stbox(p.Geom, i.Instant) AND
  valueAtTimestamp(t.Trip, i.Instant) = p.Geom
ORDER BY p.PointId, i.InstantId, v.Licence;
```

12. Which vehicles met at a point from `Points1` at an instant from `Instants1`?

```
SELECT DISTINCT p.PointId, p.Geom, i.InstantId, i.Instant,
  v1.Licence AS Licence1, v2.Licence AS Licence2
FROM Trips t1, Vehicles v1, Trips t2, Vehicles v2, Points1 p, Instants1 i
WHERE t1.VehicleId = v1.VehicleId AND t2.VehicleId = v2.VehicleId AND
  t1.VehicleId < t2.VehicleId AND t1.Trip @> stbox(p.Geom, i.Instant) AND
  t2.Trip @> stbox(p.Geom, i.Instant) AND
  valueAtTimestamp(t1.Trip, i.Instant) = p.Geom AND
  valueAtTimestamp(t2.Trip, i.Instant) = p.Geom
ORDER BY p.PointId, i.InstantId, v1.Licence, v2.Licence;
```

13. Which vehicles travelled within one of the regions from `Regions1` during the periods from `Periods1`?

```
SELECT DISTINCT r.RegionId, p.PeriodId, p.Period, v.Licence
FROM Trips t, Vehicles v, Regions1 r, Periods1 p
WHERE t.VehicleId = v.VehicleId AND t.trip && stbox(r.Geom, p.Period) AND
  ST_Intersects(trajectory(atTime(t.Trip, p.Period)), r.Geom)
ORDER BY r.RegionId, p.PeriodId, v.Licence;
```

14. Which vehicles travelled within one of the regions from `Regions1` at one of the instants from `Instants1`?

```
SELECT DISTINCT r.RegionId, i.InstantId, i.Instant, v.Licence
FROM Trips t, Vehicles v, Regions1 r, Instants1 i
WHERE t.VehicleId = v.VehicleId AND t.Trip && stbox(r.Geom, i.Instant) AND
  ST_Contains(r.Geom, valueAtTimestamp(t.Trip, i.Instant))
ORDER BY r.RegionId, i.InstantId, v.Licence;
```

15. Which vehicles passed a point from `Points1` during a period from `Periods1`?

```
SELECT DISTINCT pt.PointId, pt.Geom, pr.PeriodId, pr.Period, v.Licence
FROM Trips t, Vehicles v, Points1 pt, Periods1 pr
WHERE t.VehicleId = v.VehicleId AND t.Trip && stbox(pt.Geom, pr.Period) AND
  ST_Intersects(trajectory(atTime(t.Trip, pr.Period)), pt.Geom)
ORDER BY pt.PointId, pr.PeriodId, v.Licence;
```

16. List the pairs of licences for vehicles, the first from `Licences1`, the second from `Licences2`, where the corresponding vehicles are both present within a region from `Regions1` during a period from `QueryPeriod1`, but do not meet each other there and then.

```
SELECT p.PeriodId, p.Period, r.RegionId, l1.Licence AS Licence1, l2.Licence AS Licence2
FROM Trips t1, Licences1 l1, Trips t2, Licences2 l2, Periods1 p, Regions1 r
WHERE t1.VehicleId = l1.VehicleId AND t2.VehicleId = l2.VehicleId AND
  l1.Licence < l2.Licence AND t1.Trip && stbox(r.Geom, p.Period) AND
  t2.Trip && stbox(r.Geom, p.Period) AND
  ST_Intersects(trajectory(atTime(t1.Trip, p.Period)), r.Geom) AND
  ST_Intersects(trajectory(atTime(t2.Trip, p.Period)), r.Geom) AND
  aDisjoint(atTime(t1.Trip, p.Period), atTime(t2.Trip, p.Period))
ORDER BY PeriodId, RegionId, Licence1, Licence2;
```

17. Which point(s) from `Points` have been visited by a maximum number of different vehicles?

```
WITH PointCount AS (
  SELECT p.PointId, COUNT(DISTINCT t.VehicleId) AS Hits
  FROM Trips t, Points p
  WHERE ST_Intersects(trajectory(t.Trip), p.Geom)
  GROUP BY p.PointId )
SELECT PointId, Hits
FROM PointCount AS p
WHERE p.Hits = ( SELECT MAX(Hits) FROM PointCount );
```